

ALU

2. Derive the input equations (X_i, Y_i and Z_i) for the parallel adders to be used in the ALU which satisfies the following functional design specification. (12)

S_2	S_1	C_{in}	Required Functions
0	0	0	$F = AB + C$
0	0	1	$F = AB + C + 1$
0	1	0	$F = AB$
0	1	1	$F = AB + 1$
1	0	x	$F = (AB)'$
1	1	x	$F = AB$

$$X_i = AB$$

$$Z_{in} = S_2' C_{in}$$

$$Y_i = S_2' S_1' C + S_2 S_1'$$

$$A-B = A + \bar{B} + 1$$

$$A-B-1 = A + \bar{B}$$

S_3	S_2	S_1	S_0	Required Functions	X_i	Y_i	Z_i
0	0	0	0	$F = A$	A	0	0
0	0	0	1	$F = A - 1$	A	all 1	0
0	0	1	0	$F = A + B + 1$	A	B	1
0	0	1	1	$F = A + B$	A	B	0
0	1	0	0	$F = A - B$	A	$\bar{B}$	1
0	1	0	1	$F = A - B - 1$	A	$\bar{B}$	0
0	1	1	0	$F = A + 1$	A	0	1
0	1	1	1	$F = A$	A	all 1	1
1	0	0	x	$F = A'$	$\bar{A}$	0	0
1	0	1	x	$F = AB'$	A	$\bar{B}$	0
1	1	0	x	$F = A \odot B$ [XNOR]	A	$\bar{B}$	0
1	1	1	x	$F = A \vee B$ [OR]	A	$\bar{A}B$	0

$$\begin{aligned} A \oplus Y_i &= AB' \\ \Rightarrow Y_i &= AB' \oplus A \\ &= AB' \cdot A' + (A' + B)A \\ &= AB \end{aligned}$$

(b) Howard wants to demonstrate his engineering skills, but Sheldon sets a challenging constraint for him. Howard is tasked with designing a 4-bit Arithmetic Logic Unit (ALU), and the constraint is that he cannot use any parallel adder. Instead, he is only allowed to use parallel subtractors and logic gates for the ALU design. The ALU should have three selection variables, denoted as S_0, S_1 , and S_2 . These selection variables control the ALU to perform various operations on two 4-bit inputs, Labelled as A and B. Your task is to help Howard design the 4-bit Arithmetic Logic Unit (ALU) so that it can do the following operations:

(14)

$$A-B = A \oplus B$$

S_2	S_1	S_0	Operation	X_i	Y_i	Z_i
0	0	0	A - B	A	B	0
0	0	1	A + B	A	B	1
0	1	0	-A - B - 2	$\bar{A}$	B	1
0	1	1	B - A	$\bar{A}$	$\bar{B}$	0
1	0	0	A XOR B	A	B	0
1	1	0	A AND B	A	$\bar{A}B$	0

$$X_i = S_2' S_1' A' + S_2' S_1' A + S_2 A$$

$$Y_i = S_0 B' + S_0' B + S_1 S_2' S_0' AB'$$

$$Z_i = S_2' S_1 + S_2' S_1' S_0$$

$$A \oplus Y_i = AB$$

$$\Rightarrow Y_i = AB \oplus A$$

$$= AB \cdot \bar{A} + \bar{A}B \cdot A$$

$$= (\bar{A} + B) \cdot A = \bar{B}A$$

$$A+B = A - (-B)$$

$$= A - (\bar{B} + 1)$$

$$= A - \bar{B} - 1$$

$$-A-B-2$$

$$= \bar{A} + 1 - B - 2$$

$$= \bar{A} - B - 1$$

$$B - A$$

$$= -A - (-B)$$

$$= -A - (\bar{B} + 1)$$

$$= \bar{A} + 1 - \bar{B} - 1 = \bar{A} - \bar{B}$$

$$A \oplus Y_i = A \odot B$$

$$\Rightarrow Y_i = (A \odot B) \oplus A$$

$$= (A \odot B) \bar{A} + \overline{A \odot B} \cdot A$$

$$= (\bar{A}B + AB) \bar{A} + (A\bar{B} + \bar{A}B) A$$

$$= \bar{A}\bar{B} + A\bar{B} = \bar{B}$$

$$A \oplus Y_i = A + B$$

$$\Rightarrow Y_i = (A+B) \oplus A$$

$$= (A+B) \bar{A} +$$

$$(\bar{A} + B)A$$

$$= \bar{A}B + \bar{A} \cdot B \cdot A$$

$$= \bar{A}B$$

A & B input

S_2	S_1	S_0	Operation	X_i	Y_i	Z_i
0	0	0	Addition	A	B	0
0	0	1	Transfer A	A	all 1	1
0	1	0	Add 1's complement of B to A	A	$\bar{B}$	0
0	1	1	Increment A	A	0	1
1	0	0	XOR	A	B	0
1	0	1	NOT	A	all 1	0
1	1	0	AND	A	$\bar{A}B$	0
1	1	1	OR	A	$\bar{A}B$	0

$$A \oplus Y_i = A'$$

$$\Rightarrow Y_i = A' \oplus A = A + A' = 1$$

$$A \oplus Y_i = AB$$

$$Y_i = AB \oplus A$$

$$= (\bar{A} + B)A = \bar{A}B$$

$$A \oplus Y_i = A + B$$

$$Y_i = \bar{A}B$$

$$\begin{aligned} &B'S_2'(S_1' + S_1A) \\ &+ S_2S_1S_0AB \end{aligned}$$

$$Z_i = S_3'S_2S_1 + S_3'S_2S_1'S_0' + S_2'S_1S_0'$$

$$= S_3'S_2S_1 + S_3'S_1'(S_1 \oplus S_2)$$

S_3	S_2	S_1	S_0	Required Functions	X_i	Y_i	Z_i
0	0	0	0	$F = A'$	A'	0	0
0	0	0	1	$F = A' + 1$	A'	0	1
0	0	1	0	$F = A' + B$	A'	B	0
0	0	1	1	$F = A' + B + 1$	A'	B	1
0	1	0	0	$F = A' - B - 1$	A'	B'	0
0	1	0	1	$F = A' - B$	A'	B'	1
0	1	1	0	$F = A' - 1$	A'	all 1	0
0	1	1	1	$F = A'$	A'	all 1	1
1	0	0	x	$F = (AB)'$ [NAND]	A'	$\bar{A}B$	0
1	0	1	x	$F = A \odot B$ [XNOR]	A'	B	0
1	1	0	x	$F = A \oplus B$ [XOR]	A'	B'	0
1	1	1	x	$F = A \vee B$ [OR]	A'	$\bar{A}B$	0

Table 2a: Required Functional design specification

$$X_i = A', Z_i = S_0, Y_i = S_3'S_2'S_1B + S_3'S_2'S_1'B' + S_3S_2S_1B + S_3S_2S_1'B'$$

$$S_3S_2S_1AB' + S_3S_2S_1(A+B')$$

$$= S_3'S_1B + S_3S_1'B' + S_3S_2S_1AB' + S_3S_2S_1A + S_3S_2S_1B'$$



$$A' \oplus (A+B) = A'(A+B) + A'(A'B) = A'AB + A'B'$$

$$= A + A'B'$$

$$A' \oplus (A \odot B) = A(A \odot B) + A'(A \odot B)$$

$$= (A'AB + A'B) + A'(A'B + AB)$$

$$= A + B'$$

$$A' \oplus Y_i = A'B'$$

$$A' \oplus Y_i = \bar{A} \odot B$$

$$\Rightarrow Y_i = (A' + B') \oplus A'$$

$$= (A' + B') \cdot A + A'AB$$

$$= AB'$$

$$\Rightarrow Y_i = (\bar{A} \odot B) \oplus A'$$

$$= \bar{A} \odot B \cdot A + (A \odot B) \cdot A'$$

$$= (\bar{A}B + AB)A + (\bar{A}B + \bar{A}B)A'$$

$$= AB + \bar{A}B = B$$

I ALU

Floating-Point Addition Practice Set

10 Problems – Each in Its Own Custom Bit Format | Medium–Hard

Every problem below uses a **different** sign-exponent-fraction layout and a different exponent bias – read the format line carefully before you start each one. In every format: numbers are normalized with an implicit leading 1, and you should keep 3 extra working bits during addition – Guard (G), Round (R), Sticky (S) – positioned immediately after the fraction field. Round using round-to-nearest, ties-to-even:

• $G=0 \rightarrow$ round down. • $G=1$ and $(R=1$ or $S=1) \rightarrow$ round up. • $G=1, R=0, S=0$ (exact tie) $\rightarrow$ round up only if it makes the fraction's last bit 0 (even).

If rounding overflows the fraction field past all-1s, shift right once more and increment the exponent again.

For every problem, show: (1) each operand's normalized binary form and its encoding in that problem's own format, (2) the exponent difference and aligned mantissa with G/R/S identified, (3) the mantissa sum, (4) normalization performed (if any) and why, (5) the rounding decision and why, and (6) the final packed bit pattern and its decimal value.

$$\begin{aligned} 12 &= 1100_2 = 1.100000 \times 2^3 \text{ --- ①} & \Delta \text{exponent} &= 3-1 = 2 & \text{①} + \text{②} \Rightarrow & 12+35 = 1.111100 \times 2^3 \\ 3.5 &= 11.1_2 = 1.110000 \times 2^1 \gg 2 & & = 2 & & \Rightarrow 15.5 = 1111.1_2 \\ & & & & & = 0.011100 \times 2^3 \text{ --- ②} \end{aligned}$$

Problem 1

Sign	Exponent	Fraction	Bias	Total bits
1 bit	4 bits	6 bits	7	11

Compute $12 + 3.5$ in this format.

Hint: Straightforward alignment – check whether rounding is even needed here.

Problem 2

Sign	Exponent	Fraction	Bias	Total bits
1 bit	5 bits	4 bits	15	10

Compute $28 + 4$ in this format.

Hint: Watch the mantissa sum carefully – does it stay below 2.0?

$$\begin{aligned} 28 &= 11100_2 = 1.1100 \times 2^4 \text{ --- ①} & \text{①} + \text{②} \Rightarrow & 10.0000 \times 2^4 \\ 4 &= 100_2 = 1.0000 \times 2^0 \gg 4 = 0.0100 \times 2^4 \text{ --- ②} & & = 1 \times 2^5 \\ & & & = 100000_2 \end{aligned}$$

Problem 3

Sign	Exponent	Fraction	Bias	Total bits
1 bit	3 bits	8 bits	3	12

Compute $16 + 0.05859375$ in this format.

Hint: The exponent gap pushes several bits of the smaller operand past the fraction field.

$$\begin{aligned} 16 &= 10000_2 = 1.00000000 \times 2^4 = 1.000000000000 \times 2^4 \text{ --- ①} & G^* (R \vee S \vee LSP) & \\ 0.05859375 &= 0.00001111 \times 2^0 = 0.000000001111 \times 2^4 \text{ --- ②} & & = 1 \rightarrow \text{Round up} \\ & & & \text{G R S} \end{aligned}$$

$$\text{①} + \text{②} \Rightarrow 1.00000001 \times 2^4 = 10000.00010000_2$$

Problem 4

Sign	Exponent	Fraction	Bias	Total bits
1 bit	6 bits	5 bits	40	12

Compute $4.125 + 0.0625$ in this format.

Hint: Bias here is unusually large – don't forget to add it when packing the exponent.

Problem 5

Sign	Exponent	Fraction	Bias	Total bits
1 bit	4 bits	10 bits	5	15

Compute $8 + 0.00390625$ in this format.

Hint: This format has a wide fraction field – track the shift far enough to find where the only surviving bit lands.

Problem 6

Sign	Exponent	Fraction	Bias	Total bits
1 bit	7 bits	3 bits	63	11

Compute $0.28125 + (-0.25)$ in this format.

Hint: Opposite signs, equal exponents, and a very short fraction field – cancellation will be dramatic here.

Problem 7

Sign	Exponent	Fraction	Bias	Total bits
1 bit	5 bits	7 bits	20	13

Compute $3.984375 + 0.01171875$ in this format.

Hint: The starting fraction is already all 1s before you even add anything.

Problem 8

Sign	Exponent	Fraction	Bias	Total bits
1 bit	3 bits	10 bits	2	14

Compute $32 + 0.00048828125$ in this format.

Hint: A very large exponent gap in a format with a small exponent field – think about what survives alignment.

Problem 9

Sign	Exponent	Fraction	Bias	Total bits
1 bit	6 bits	6 bits	50	13

Compute $-4 + (-0.25)$ in this format.

Hint: Both operands negative – confirm the sign handling doesn't change your alignment/rounding steps.

$$\begin{aligned}
 -4 &= -100_2 = -1.000000 \times 2^2 \quad \text{--- ①} & 2 - (-2) &= 4 \\
 -0.25 &= -0.01_2 = -0.010000 \times 2^0 = -1.000000 \times 2^{-2} \gg 4 = -0.000100 \times 2^2 \quad \text{--- ②} \\
 \text{①} + \text{②} &\Rightarrow -1.000100 \times 2^2 = -100.010000_2 = -4.25
 \end{aligned}$$

Problem 10

Sign	Exponent	Fraction	Bias	Total bits
1 bit	4 bits	4 bits	9	9

Compute $15.5 + 0.375$ in this format.

Hint: The smallest format in the set – every bit matters, and rounding may trigger a second normalization.

$$\begin{aligned}
 15.5 &= 1111.1_2 = 1.1111 \times 2^4 \quad \text{--- ①} & \Delta \text{exponent} &= 3-0 = 3 \\
 0.375 &= 0.0110 \times 2^0 \gg 3 = 0.000011 \times 2^3 \quad \text{--- ②} & & \\
 & & & = 1.111011 \times 2^3
 \end{aligned}$$

Note: a companion solutions document works through all ten problems in full, bit-by-bit, in each problem's own format, including the exact guard/round/sticky bits and a final decimal check against the true mathematical sum.

Floating-Point Addition Practice Set – Set 2

10 Problems – Each in Its Own Custom Bit Format | Medium–Hard

Every problem below uses a **different** sign-exponent-fraction layout and a different exponent bias – read the format line carefully before you start each one. In every format: numbers are normalized with an implicit leading 1, and you should keep 3 extra working bits during addition – Guard (G), Round (R), Sticky (S) – positioned immediately after the fraction field. Round using round-to-nearest, ties-to-even:

• $G=0 \rightarrow$ round down. • $G=1$ and $(R=1 \text{ or } S=1) \rightarrow$ round up. • $G=1, R=0, S=0$ (exact tie) $\rightarrow$ round up only if it makes the fraction's last bit 0 (even).

If rounding overflows the fraction field past all-1s, shift right once more and increment the exponent again.

For every problem, show: (1) each operand's normalized binary form and its encoding in that problem's own format, (2) the exponent difference and aligned mantissa with G/R/S identified, (3) the mantissa sum, (4) normalization performed (if any) and why, (5) the rounding decision and why, and (6) the final packed bit pattern and its decimal value.

Problem 1

Sign	Exponent	Fraction	Bias	Total bits
1 bit	5 bits	6 bits	16	12

Compute $40 + 12$ in this format.

Hint: Straightforward alignment – check whether rounding is even needed here.

Problem 2

Sign	Exponent	Fraction	Bias	Total bits
1 bit	4 bits	8 bits	11	13

Compute $14 + 2$ in this format.

Hint: Watch the mantissa sum carefully – does it stay below 2.0?

Problem 3

Sign	Exponent	Fraction	Bias	Total bits
1 bit	6 bits	4 bits	29	11

Compute $64 + 3$ in this format.

Hint: The exponent gap pushes several bits of the smaller operand past the fraction field.

Problem 4

Sign	Exponent	Fraction	Bias	Total bits
1 bit	3 bits	9 bits	4	13

$100.000000100_2 = 1.000000001 \times 2^2 - \textcircled{1}$
 $\textcircled{1} + \textcircled{2} \Rightarrow 1.00000001100 \times 2^2$
 $1.00000010 \times 2^0 = 100.000001000$
 $\hookrightarrow 0.00000010 \times 2^0 \gg 2 = 0.00000000100 \times 2^2 - \textcircled{2}$

Compute $4.0078125 + 0.00390625$ in this format.

Hint: Bias here is unusually large – don't forget to add it when packing the exponent.

Problem 5

Sign	Exponent	Fraction	Bias	Total bits
1 bit	5 bits	9 bits	17	15

Compute $16 + 0.015625$ in this format.

Hint: This format has a wide fraction field – track the shift far enough to find where the only surviving bit lands.

Problem 6

Sign	Exponent	Fraction	Bias	Total bits
1 bit	8 bits	2 bits	127	11

Compute $0.15625 + (-0.125)$ in this format.

Hint: Opposite signs, equal exponents, and a very short fraction field – cancellation will be dramatic here.

Problem 7

Sign	Exponent	Fraction	Bias	Total bits
1 bit	4 bits	7 bits	6	12

Compute $7.96875 + 0.0234375$ in this format.

Hint: The starting fraction is already all 1s before you even add anything.

Problem 8

Sign	Exponent	Fraction	Bias	Total bits
1 bit	6 bits	3 bits	31	10

Compute $16 + 0.0625$ in this format.

Hint: A very large exponent gap in a format with a small exponent field – think about what survives alignment.

Problem 9

Sign	Exponent	Fraction	Bias	Total bits
1 bit	5 bits	5 bits	14	11

Compute $-8 + (-2)$ in this format.

Hint: Both operands negative – confirm the sign handling doesn't change your alignment/rounding steps.

Problem 10

Sign	Exponent	Fraction	Bias	Total bits
1 bit	3 bits	5 bits	3	9

Compute $7.875 + 0.09375$ in this format.

Hint: The smallest format in the set – every bit matters, and rounding may trigger a second normalization.

Note: a companion solutions document works through all ten problems in full, bit-by-bit, in each problem's own format, including the exact guard/round/sticky bits and a final decimal check against the true mathematical sum.

Floating-Point Addition Practice Set – Set 3

5 Problems – Unbiased (Two's-Complement) Exponent | Medium–Hard

No bias this time. In every format below, the exponent field stores the **true exponent directly, in two's-complement** – there is no bias to add or subtract. An exponent field of width k can represent true exponents from -2^{k-1} to $2^{k-1}-1$. To decode a stored exponent field, read it as an ordinary two's-complement signed integer (if the leading bit is 1, the value is negative). To encode a true exponent, take its two's-complement representation in the given field width.

As before: numbers are normalized with an implicit leading 1, and you should keep 3 extra working bits during addition – Guard (G), Round (R), Sticky (S) – positioned immediately after the fraction field. Round using round-to-nearest, ties-to-even:

• $G=0 \rightarrow$ round down. • $G=1$ and ($R=1$ or $S=1$) $\rightarrow$ round up. • $G=1, R=0, S=0$ (exact tie) $\rightarrow$ round up only if it makes the fraction's last bit 0 (even).

If rounding overflows the fraction field past all-1s, shift right once more and increment the exponent again.

For every problem, show: (1) each operand's normalized binary form and its encoding in that problem's own format (remember: two's-complement exponent, no bias), (2) the exponent difference and aligned mantissa with G/R/S identified, (3) the mantissa sum, (4) normalization performed (if any) and why, (5) the rounding decision and why, and (6) the final packed bit pattern and its decimal value.

Problem 1

Sign	Exponent	Fraction	Exponent range	Total bits
1 bit	4 bits (two's complement)	6 bits	–8 to 7	11

Compute $5 + 1.5$ in this format.

Hint: Straightforward alignment – both exponents are non-negative here, so it's a good warm-up for the two's-complement encoding.

Problem 2

Sign	Exponent	Fraction	Exponent range	Total bits
1 bit	5 bits (two's complement)	7 bits	–16 to 15	13

Compute $12 + 6$ in this format.

Hint: Watch the mantissa sum carefully – does it stay below 2.0?

Problem 3

Sign	Exponent	Fraction	Exponent range	Total bits
1 bit	4 bits (two's complement)	8 bits	-8 to 7	13

Compute $8 + 0.0234375$ in this format.
 $8 = 1000 = 1.00000000 \times 2^3$ ② $0.0234375 = 0.00000110 \times 2^0 \gg 3$ Round = 1
 $= 0.0000000110 \times 2^3$ ① $①+② \Rightarrow 1.00000001 \times 2^3 = 1000.0000100 = 8.03125$

Hint: The exponent gap pushes bits of the smaller operand past the fraction field. Note that the smaller operand's true exponent is negative – encode it carefully in two's complement.

Problem 4

Sign	Exponent	Fraction	Exponent range	Total bits
1 bit	6 bits (two's complement)	4 bits	-32 to 31	11

Compute $0.53125 + 0.015625$ in this format.

Hint: This lands on an exact tie. Both operands here have negative true exponents – double-check your two's-complement encoding before you align.

Problem 5

Sign	Exponent	Fraction	Exponent range	Total bits
1 bit	5 bits (two's complement)	3 bits	-16 to 15	9

Compute $0.0859375 + (-0.0625)$ in this format.

Hint: Opposite signs, equal (negative) exponents – expect significant cancellation and a left-shift. Watch how far the true exponent drops.

Note: a companion solutions document works through all five problems in full, bit-by-bit, including the two's-complement exponent decode/encode step, the exact guard/round/sticky bits, and a final decimal check against the true mathematical sum.

MIPS

(c) Translate function f (defined below) into MIPS assembly language. If you need to use registers \$t0 through \$t7, use the lower-numbered registers first. Assume the function declaration for func is "int func(int a, int b);". The code for function f is as follows:

```
int f (int a, int b, int c, int d) {
    return func(func(a,b), c+d);
}
```

*one for return ad
storing*

```
f: addi $sp, $sp, -8
   sw $a0, 4($sp)
   sw $a2, 0($sp)
   jal func
   lw $a2, 0($sp)
   add $a1, $a2, $a3
   addi $a0, $v0, $zero
   jal func
   lw $ra, 4($sp)
   addi $sp, $sp, 8
   jr $ra
```

```
int testFunc(int a, int b, int c) {
    int i = 0, result = 0;
    do {
        if (i > b)
            result += c;
        i++;
    } while (i < a);
    return result;
}
```

```
testFunc: addi $sp, $sp, -8
          sw $s1, 4($sp)
          sw $s2, 0($sp)
          add $s1, $zero, $zero
          add $s2, $zero, $zero
```

```
Loop:
      slt $t0, $a1, $s1
      beq $t0, $zero, Skip
      add $s2, $s2, $a2
```

```
Skip:
      addi $s1, $s1, 1
      slt $t1, $s1, $a0
      bne $t1, $zero, Loop
      addi $v0, $s2
```

```
      lw $s1, 4($sp)
      lw $s2, 0($sp)
      addi $sp, $sp, 8
      jr $ra
```

(d) Consider the following C code which calculates a^b (a raised power b),
int fun (int a, int b)

```
{
    if (b == 0)
        return 0;
    else if (b % 2 == 0)
        return fun (a*a, b/2);
    return fun (a*a, b/2) * a;
}
```

Write down the equivalent MIPS assembly code for it. Use a minimal number of MIPS instructions.

```
fun: addi $sp, $sp, -12
     sw $a0, 8($sp)
     sw $a1, 4($sp)
     sw $ra, 0($sp)
     beq $a1, $zero, if
     andi $t0, $a1, 1
     beq $t0, $zero, else_if
     mul $a0, $a0, $a0
```

snl \$a1,\$a1,1

jal fun

lw \$a0, 8(\$sp)

lw \$a1, 4(\$sp)

mul \$v0, \$v0, \$a0

j Done

if:

addi \$v0, \$zero, 0

j Done

else_if: andi \$t0, \$a1, 1

beq \$t0, \$zero, else_if

mul \$a0, \$a0, \$a0

snl \$a1,\$a1,1

jal fun

lw \$a0, 8(\$sp)

lw \$a1, 4(\$sp)

j Done

Done: lw \$ra, 0(\$sp)

addi \$sp, \$sp, 12

jr \$ra

```
int recurseSum(int n) {  
    if (n < 1) return 0;  
    else return n + recurseSum(n-1);  
}
```

recurseSum: addi \$sp, \$sp, -8

lw \$a0, 4(\$sp)

lw \$ra, 0(\$sp)

slti \$t0, \$a0, 1

beq \$t0, \$zero, Recursive

addi \$v0, \$zero, 0

jr \$ra

Recursive:

addi \$a0, \$a0, -1

jal recurseSum

lw \$a0, 4(\$sp)

add \$v0, \$v0, \$a0

addi \$sp, \$sp, 8

jr \$ra

(a) Produce the compiled MIPS code for the following C function. Assume that addresses of `dest`, `source` in `$a0`, and `$a1`. Use a minimal number of MIPS assembly instructions.

```

20 void rstrcpy (char *dest, char *source)
{
    if (*source)
    {
        *dest++=*source++;
        rcopy (dest, source);
    }
    else
        *dest='\0';
}

```

```

21 void strcpy (char x[], char y[])
{
    int i;
    i = 0;
    while ((x[i] = y[i]) != '\0') /* copy & test byte */
        i += 1;
}

```

What is the MIPS assembly code?

strcpy: addi \$sp, \$sp, -4

lw \$s0, 0(\$sp)

addi \$s0, \$zero, 0

Loop: add \$t0, \$s0, \$a1

lb \$t0, 0(\$t0)

beq \$t0, \$zero, End

add \$t1, \$s0, \$a0

sb \$t0, 0(\$t1)

addi \$s0, \$s0, 1

j Loop

End: lw \$ra, 0(\$sp)

addi \$sp, \$sp, 4

jr \$ra

rcopy: lb \$t0, 0(\$a1)

beq \$t0, \$zero, Else

addi \$sp, \$sp, -4

sw \$ra, 0(\$sp)

sb \$t0, 0(\$a0)

addi \$a0, \$a0, 1

addi \$a1, \$a1, 1

jal rcopy

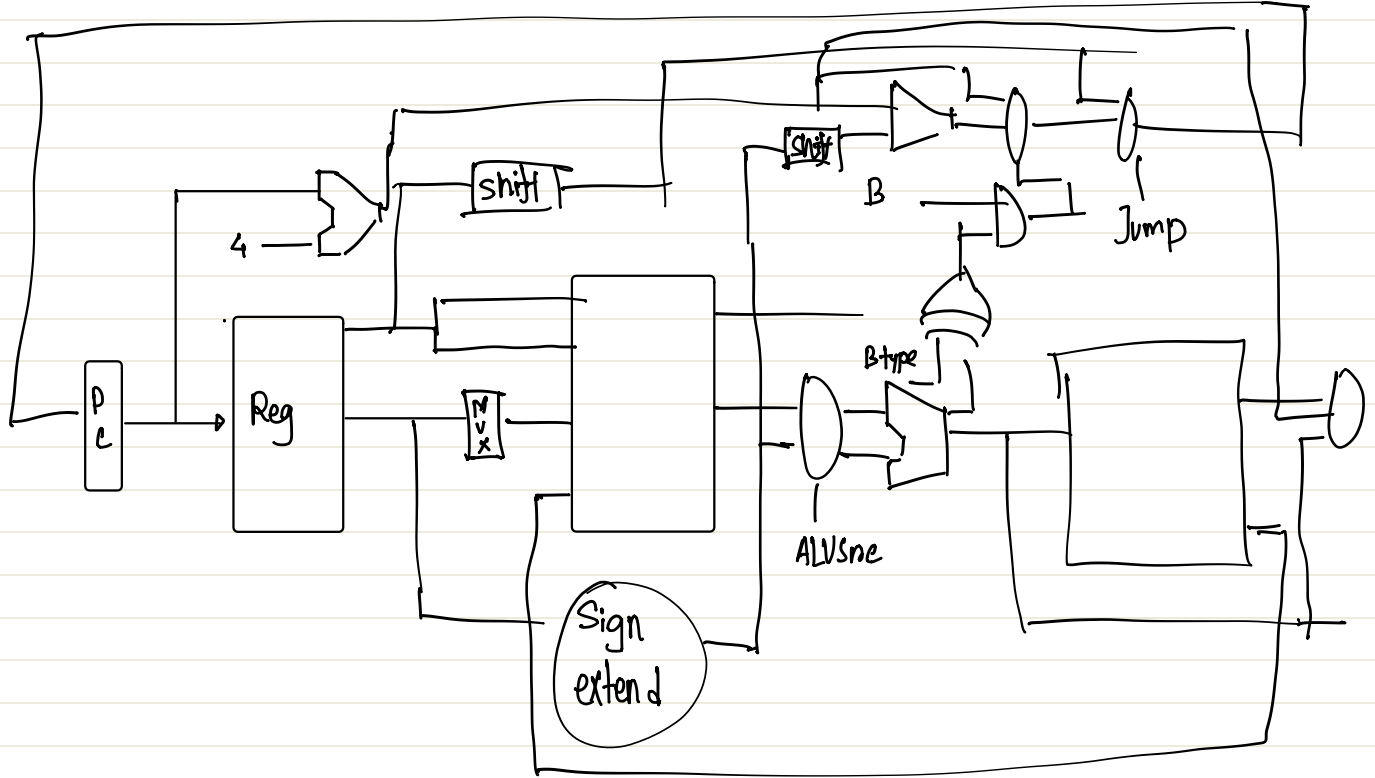
lw \$ra, 0(\$sp)

addi \$sp, \$sp, 4

jr \$ra

Else: sb \$zero, 0(\$a0)

jr \$ra



Quiz Study Guide

Concepts from the lectures, with practice — then every past paper solved

Part I — Concepts. ALU design • computer arithmetic & floating point • MIPS instruction set & assembly • single-cycle datapath & control • pipelining • performance • cache.

Each topic is taught from the class slides and ends with practice problems and solutions.

Part II — Past papers, solved in full. The CSE 306 quiz paper, a senior's recollection of another CSE 306 quiz, CT-1 to CT-4, and four mock papers.

Contents

How to use this guide	3
I Concepts from the lectures, with practice	3
1 ALU design	3
1.1 What an ALU is	3
1.2 Adders: the building block	4
1.3 The arithmetic circuit	4
1.4 A design example from the lecture	5
1.5 The logic circuit and how the two are combined	5
1.6 The more efficient design	6
1.7 Status flags	6
1.8 The derivation recipe	7
1.9 Practice	7
2 Computer arithmetic and floating point	9
2.1 Integer addition, subtraction and overflow	9
2.2 Multiplication in one paragraph	9
2.3 Floating-point representation	9
2.4 Range and precision	10
2.5 Special values	10
2.6 Converting	10
2.7 Floating-point addition	11
2.8 Practice	11
3 The MIPS instruction set and assembly	13
3.1 Registers and the calling convention	13
3.2 The three instruction formats	13
3.3 Opcodes to memorise	14
3.4 Addressing modes and target calculations	14
3.5 Translating C	14
3.6 Procedures and the stack	15
3.7 Practice	15
4 The single-cycle datapath and control	17
4.1 The building blocks	17
4.2 The control signals	17
4.3 How each instruction uses the datapath	20
4.4 Implementing the control unit	30
4.5 Practice	30
5 Pipelining	31
5.1 The idea	31
5.2 Hazards	32
5.3 Forwarding and stalls	32
5.4 Branch hazards	32
5.5 Practice	33
6 Performance	33
6.1 Practice	34
7 Cache (short)	34

7.1 Practice	35
II Past papers, solved	35
8 CSE 306 Quiz Examination (45 marks, 65 minutes)	35
9 CSE 306 quiz, as recollected by a senior	44
10 CT-1 (6 July 2026, 20 marks)	45
11 CT-2 (3 August 2026, 20 marks)	46
12 CT-3 (20 marks)	47
13 CT-4 (20 marks)	48
14 Four mock papers	48
A One-page cheat sheet	50

How to use this guide

1. **Part I, one topic at a time.** Read the explanation, then do the practice problems at the end of that topic *before* looking at the solutions.
2. **Part II, as exams.** Each paper is printed as it was set, followed by a full solution. Try the paper under time pressure first.
3. **The cheat sheet** (last page) is for the final half hour.

What the quiz is likely to look like

The CSE 306 paper in Part II is about three years old, but the course (now CSE 210) has kept the same topics, and a senior's notes from a later batch match it closely. Expect roughly: 10–15 MCQs, a few fill-in-the-blanks, an ALU equation derivation, a control-bit table to fill, a set of opcodes to write, floating-point conversion/addition, datapath marking for one instruction, and short questions from each topic. CT questions may be reused where the topics overlap.

Sources and how they are cited

Every Part I section opens with a grey **Lecture** / **Textbook** line, and every Part II question with a **Concept** / **Source** line.

- **ALU slides** — ART Sir, *Arithmetic Logic Unit* (CSE 209), 23 slides.
- **Ch1 / Ch2 / Ch3 slides** — ART Sir, Patterson & Hennessy Chapters 1, 2 and 3 (*Arithmetic*, short slides).
- **Ch4 Part 1 / Part 2 slides** — RS Sir, *The Processor* (single-cycle, then pipelining).
- **Control-unit notes** — RS Sir, *Implementation of the MIPS Control Unit* (= COD Appendix D).
- **Ch5 slides** — RS Sir, *Large and Fast: Exploiting Memory Hierarchy*.
- **COD** — Patterson & Hennessy, *Computer Organization and Design*, 5th ed. (the PDF in RS Sir's folder).
- **Mano** — M. Morris Mano, *Digital Logic and Computer Design*, Chapter 9 (§9.3–9.7), cited on the ALU slides.

Slide numbers are the page numbers of your PDFs. Figure numbers were checked against the COD PDF and the figure labels printed on the slides.

Part I

Concepts from the lectures, with practice

1 ALU design

Lecture: ALU slides 2–22 (ART Sir); Assignment 1 • **Textbook:** Mano, *Digital Logic and Computer Design*, Ch. 9 (§9.3–9.7)

1.1 What an ALU is

Lecture: ALU slide 2 • **Textbook:** Mano Fig 9-5 (block diagram of a 4-bit ALU)

An **Arithmetic Logic Unit** performs both arithmetic and logic operations on two n -bit inputs A and B . In the lecture's 4-bit block diagram (Mano, Figure 9-5) it has:

Input / output	Name	Purpose
s_2	mode select	arithmetic or logic operation
s_1, s_0	function select	which operation within the mode
C_{in}	input carry	doubles the number of arithmetic variants (e.g. $A + B$ vs $A + B + 1$)
F	output	the n -bit result
C_{out}	output carry	carry out of the most significant stage

1.2 Adders: the building block

Lecture: ALU slides 3–5 • **Textbook:** Mano Ch. 9 (parallel adder); COD Appendix B (ALU construction)

Half adder (two input bits): $S = x'y + xy' = x \oplus y$, $C = xy$.

Full adder (two bits plus a carry z):

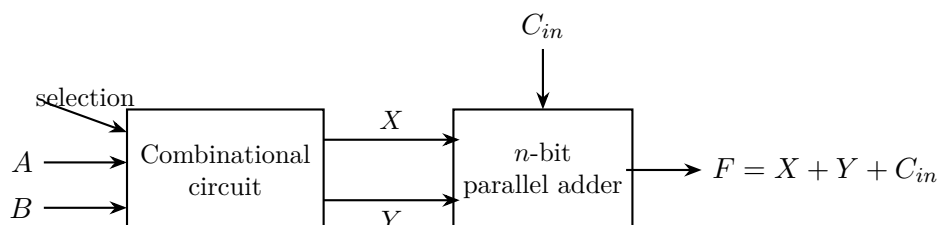
$$S = x'y'z + x'yz' + xy'z' + xyz = x \oplus y \oplus z, \quad C = xy + xz + yz$$

Parallel (ripple-carry) adder: n full adders in cascade. Stage i receives A_i , B_i and the carry C_i ; it produces S_i and C_{i+1} . C_1 is the external carry-in and C_{n+1} is C_{out} . Because every stage waits for the carry of the one before, **the carry chain is the critical path**: its delay grows linearly with n .

1.3 The arithmetic circuit

Lecture: ALU slides 6–11 • **Textbook:** Mano Figs 9-6, 9-7, 9-8 and Tables 9-1, 9-2

The central idea of the lecture: *never build separate adders and subtractors*. Put a combinational circuit in front of one parallel adder and feed it modified inputs:



With $X = A$ fixed, the eight operations of the lecture (Figure 9-6) come from choosing Y and C_{in} :

Y	C_{in}	Output	Function
B	0	$F = A + B$	addition
B	1	$F = A + B + 1$	addition with carry
$\overline{B}$	0	$F = A + \overline{B}$	A plus 1's complement of B ($A - B - 1$)
$\overline{B}$	1	$F = A + \overline{B} + 1$	subtraction ($A - B$)
0	0	$F = A$	transfer A
0	1	$F = A + 1$	increment A
all 1s	0	$F = A - 1$	decrement A
all 1s	1	$F = A$	transfer A

Table 1: The eight arithmetic operations of one parallel adder (the four choices of Y times the two of C_{in}). Mano Fig 9-6; ALU slides 6–7.

The true/complement, one/zero circuit (Mano Fig 9-7, ALU slide 8; the full 4-bit circuit is Fig

9-8, slide 9) produces the four choices of Y with just two selection lines:

$$X_i = A_i, \quad Y_i = B_i s_0 + \overline{B_i} s_1 \quad \Rightarrow$$

s_1	s_0	Y_i
0	0	0
0	1	B_i
1	0	$\overline{B_i}$
1	1	1

s_1	s_0	C_{in}	Y	Output	Function
0	0	0	0	$F = A$	transfer A
0	0	1	0	$F = A + 1$	increment A
0	1	0	B	$F = A + B$	add B to A
0	1	1	B	$F = A + B + 1$	add B to A plus 1
1	0	0	$\overline{B}$	$F = A + \overline{B}$	add 1's complement of B
1	0	1	$\overline{B}$	$F = A + \overline{B} + 1$	add 2's complement of B (subtract)
1	1	0	all 1s	$F = A - 1$	decrement A
1	1	1	all 1s	$F = A$	transfer A

Table 2: Function table of the arithmetic circuit. Mano Table 9-1; ALU slide 10.

What the output carry means (lecture Table 9-2) — this is where most carry-flag MCQs come from:

$s_1 s_0 C_{in}$	Function	$C_{out} = 1$ if	Comment
000	$F = A$	never	C_{out} always 0
001	$F = A + 1$	$A = 2^n - 1$	then $F = 0$
010	$F = A + B$	$A + B \geq 2^n$	unsigned overflow
011	$F = A + B + 1$	$A + B \geq 2^n - 1$	unsigned overflow
100	$F = A - B - 1$	$A > B$	if $C_{out} = 0$: $A \leq B$, F is 1's complement of $B - A$
101	$F = A - B$	$A \geq B$	if $C_{out} = 0$: $A < B$, F is 2's complement of $B - A$
110	$F = A - 1$	$A \neq 0$	$C_{out} = 1$ except when $A = 0$
111	$F = A$	always	C_{out} always 1

Table 3: Effect of the output carry. Mano Table 9-2; ALU slide 11. The key line: after $A - B$, a carry of 1 means *no borrow*, i.e. $A \geq B$.

1.4 A design example from the lecture

Lecture: ALU slides 12–14 • **Textbook:** Mano Fig 9-10 (4-bit adder/subtractor)

Design an adder/subtractor with one selection variable s : $s = 0$ gives $A + B$, $s = 1$ gives $A - B$ by taking the 2's complement of B .

For $s = 0$: $X = A$, $Y = B$, $C_{in} = 0$. For $s = 1$: $X = A$, $Y = \overline{B}$, $C_{in} = 1$. The truth table over (s, A_i, B_i) gives

$$X_i = A_i, \quad Y_i = B_i \oplus s, \quad C_{in} = s$$

One XOR gate per bit, and s doubles as the carry-in (Figure 9-10).

1.5 The logic circuit and how the two are combined

Lecture: ALU slides 15–16 • **Textbook:** Mano Fig 9-11 (one stage of logic circuit), Fig 9-12 (combining)

One stage of the lecture's logic circuit (Figure 9-11) computes four functions and picks one with a 4-to-1 multiplexer:

s_1	s_0	Output	Operation
0	0	$F_i = A_i + B_i$	OR
0	1	$F_i = A_i \oplus B_i$	XOR
1	0	$F_i = A_i B_i$	AND
1	1	$F_i = \overline{A_i}$	NOT

Figure 9-12 combines them: one stage of the arithmetic circuit and one stage of the logic circuit share A_i , B_i , s_1 , s_0 , and a 2-to-1 mux selected by s_2 picks the result. Simple, but it duplicates hardware.

1.6 The more efficient design

Lecture: ALU slides 17–20 • **Textbook:** Mano Table 9-3, Fig 9-13 (logic diagram of the ALU), Table 9-4

The lecture's three-step method removes the separate logic unit:

1. Design the arithmetic section on its own.
2. See which logic functions it already produces when all input carries are forced to 0. A full adder with $Z_i = 0$ gives $F_i = X_i \oplus Y_i$.
3. Modify the arithmetic circuit to obtain the logic operations that are still missing.

s_2	s_1	s_0	X_i	Y_i	C_i	$F_i = X_i \oplus Y_i$	Operation	Required
1	0	0	A_i	0	0	A_i	transfer	OR
1	0	1	A_i	B_i	0	$A_i \oplus B_i$	XOR	XOR
1	1	0	A_i	$\overline{B_i}$	0	$A_i \odot B_i$	equivalence	AND
1	1	1	A_i	1	0	$\overline{A_i}$	NOT	NOT

Table 4: Logic operations one arithmetic stage produces with its carries at 0. Mano Table 9-3; ALU slide 17.

XOR and NOT come free. OR is obtained by making $X_i = A_i + B_i$ in row 100, since then $F = (A + B) \oplus 0$. AND is obtained by making $X_i = A_i + \overline{B_i}$ in row 110, since $(A + \overline{B}) \oplus \overline{B} = AB$. The final equations are:

$$X_i = A_i + s_2 \overline{s_1} \overline{s_0} B_i + s_2 s_1 \overline{s_0} \overline{B_i}, \quad Y_i = s_0 B_i + s_1 \overline{B_i}, \quad Z_i = \overline{s_2} C_i$$

They give the twelve functions of Table 9-4: the eight arithmetic operations of Table 9-1 when $s_2 = 0$, and OR, XOR, AND, NOT when $s_2 = 1$ (with C_{in} a don't-care).

Why $Z_i = \overline{s_2} C_i$ matters

In logic mode every carry must be 0 — not just the input carry, but the carry between stages too. Otherwise a carry generated in stage i ripples into stage $i + 1$ and corrupts an OR or XOR. Forgetting this gate is the classic way to lose marks on these questions.

1.7 Status flags

Lecture: ALU slides 21–22; Assignment 1 §2 (flag rules) • **Textbook:** Mano Fig 9-14 (setting bits in a status register)

Flag	Set when	Notes
C (carry)	the output carry of the ALU is 1	after $A - B$: $C = 1$ means no borrow, $A \geq B$
S (sign)	the MSB of the result is 1	a copy of F_{n-1}
Z (zero)	the result is all 0s	one NOR gate over all result bits
V (overflow)	the carry into the MSB XOR the carry out of it	for 8 bits, $V = C_8 \oplus C_9$; meaningful only for signed (2's complement) numbers

Overflow in words: adding two positives and getting a negative, or two negatives and getting a positive. Adding a positive to a negative can never overflow.

Assignment 1's flag rules

- After **AND/OR/XOR**: C and V are cleared to 0; S and Z follow the result.
- After **NOT**: C and V are don't-cares; Z must not wrongly go from 1 to 0; S may stay or follow the result.
- Logic operations have no carry or signed-overflow meaning — that is why C and V are cleared.

1.8 The derivation recipe

Lecture: ALU slides 6–20, applied • **Textbook:** Mano Tables 9-1, 9-3, 9-4

Every “derive X_i , Y_i , Z_i ” question is solved the same way.

Five steps

1. Next to each row of the given table, write the $(X_i, Y_i, \text{carry-in})$ that produces it, using the operation table above. For logic rows, write carry = 0.
2. X_i : it is either A_i or $\overline{A_i}$. Collect the rows that complement it into a condition G , and write $X_i = A_i \oplus G$.
3. Y_i : it is one of 0, B_i , $\overline{B_i}$, 1. Write $Y_i = (\dots)B_i + (\dots)\overline{B_i} + (\dots)$ and simplify with a K-map.
4. Z_i : the carry, gated so it is 0 in logic mode, $Z_i = (\text{arithmetic mode}) \cdot C_i$, with Z_1 equal to the carry-in each arithmetic row needs.
5. Check every row by substituting a pair of values for A and B , and state the gate count.

1.9 Practice

Practice 1.1. Which Y and C_{in} give “add 2's complement of B ”? What does $C_{out} = 0$ tell you afterwards?

Solution. $Y = \overline{B}$, $C_{in} = 1$: $F = A + \overline{B} + 1 = A - B$. From Table 9-2, $C_{out} = 0$ means a borrow was needed: $A < B$, and F holds the 2's complement of $B - A$.

Practice 1.2. S_1S_0 : 00 transfer A , 01 add, 10 subtract, 11 decrement A . Derive X_i , Y_i , Z_i .

Solution. Rows: $(A, 0, 0)$, $(A, B, 0)$, $(A, \overline{B}, 1)$, $(A, 1, 0)$. So $X_i = A_i$; $Y_i = \overline{S_1}S_0B_i + S_1\overline{S_0}\overline{B_i} + S_1S_0$; carry-in only in row 10, so $C_1 = S_1\overline{S_0}$. There is no logic mode, so $Z_i = C_i$ with no gating.

Practice 1.3. M (mode) and S . Arithmetic ($M = 0$): $S = 0$ add with carry, $S = 1$ subtract. Logic ($M = 1$): $S = 0$ XOR, $S = 1$ NOT A .

Solution. Rows: $(A, B, 1)$, $(A, \overline{B}, 1)$, $(A, B, 0)$, $(A, 1, 0)$. $X_i = A_i$. $Y_i = \overline{M}\overline{S}B_i + \overline{M}S\overline{B_i} + M\overline{S}B_i + MS = \overline{S}B_i + \overline{M}S\overline{B_i} + MS$. Both arithmetic rows need carry-in 1, so $Z_1 = \overline{M}$, and $Z_i = \overline{M}C_i$.

Practice 1.4. $c_2c_1c_0$: 000 subtract with borrow, 001 decrement A , 010 add, 011 increment A , $1xx$ AND.

Solution. Arithmetic rows: $(A, \overline{B}, 0)$, $(A, 1, 0)$, $(A, B, 0)$, $(A, 0, 1)$. So $X_i = A_i$; $Y_i = \overline{c_2} \overline{c_1} \overline{c_0} \overline{B_i} + \overline{c_2} \overline{c_1} c_0 + \overline{c_2} c_1 \overline{c_0} B_i$; $C_1 = \overline{c_2} c_1 c_0$; $Z_i = \overline{c_2} C_i$. AND cannot come out of the adder with $Z = 0$ (that gives XOR), so add a 4-bit AND gate and a final 2-to-1 mux selected by c_2 — or use the $X_i = A_i + \overline{B_i}$, $Y_i = \overline{B_i}$ trick of §1.6.

Practice 1.5. Your own Assignment-1 specification (Section C2, Group 5): $cs_2cs_1cs_0 = 000$ increment A , 010 add with carry, 100 transfer A , 110 add, $x01$ XOR, $x11$ NOT. Derive X_i , Y_i , Z_i .

Solution. First notice the structure: cs_0 is the mode bit (0 arithmetic, 1 logic); in arithmetic mode cs_2 chooses the carry-in and cs_1 chooses whether B is added.

cs_2	cs_1	cs_0	X_i	Y_i	carry	F
0	0	0	A_i	0	1	$A + 1$
0	1	0	A_i	B_i	1	$A + B + 1$
1	0	0	A_i	0	0	A
1	1	0	A_i	B_i	0	$A + B$
x	0	1	A_i	B_i	0	$A \oplus B$
x	1	1	A_i	1	0	$\overline{A}$

X_i is never complemented: $X_i = A_i$. Y_i against (cs_1, cs_0) : $00 \rightarrow 0$, $10 \rightarrow B$, $01 \rightarrow B$, $11 \rightarrow 1$, so

$Y_i = (cs_1 \oplus cs_0)B_i + cs_1cs_0$. The carry ripples only when $cs_0 = 0$, and the LSB carry-in is 1

exactly when $cs_2 = 0$: $Z_i = \overline{cs_0} C_i$, $Z_1 = \overline{cs_0} \overline{cs_2}$.

Check with $A = 0110$, $B = 0011$: $000 \rightarrow 0111$, $010 \rightarrow 1010$, $100 \rightarrow 0110$, $110 \rightarrow 1001$, $x01 \rightarrow 0101$, $x11 \rightarrow 1001$ — all six correct.

Practice 1.6. Compute C, S, Z, V for these 4-bit operations: (a) $0111 + 0001$ (b) $1000 - 0001$ (c) $1111 + 0001$ (d) $0011 - 0101$.

Solution.

	Result	C	S	Z	V	Why
(a)	1000	0	1	0	1	$+7 + 1 = +8$ does not fit in 4-bit 2's complement
(b)	0111	1	0	0	1	$-8 - 1 = -9$ does not fit; C=1 because $8 \geq 1$ unsigned (no borrow)
(c)	0000	1	0	1	0	$-1 + 1 = 0$ fits
(d)	1110	0	1	0	0	$3 - 5 = -2$ fits; C=0 because $3 < 5$ (a borrow)

Practice 1.7. An 8-bit ripple adder: design X has sum delay 3 and carry delay 2 gate-delays per stage; design Y has sum delay 2 and carry delay 3. Which is faster, and by how much?

Solution. The carry passes through all 8 stages; the sum is paid once, at the last stage. X: $8 \times 2 + 3 = 19$. Y: $8 \times 3 + 2 = 26$. Design X (fast carry) is faster by 7 gate delays. The rule: **always optimise the carry.**

Practice 1.8. Why does $Y_i = B_i s_0 + \overline{B_i} s_1$ give all 1s when $s_1 s_0 = 11$, and what operation results with $C_{in} = 0$?

Solution. $Y_i = B_i + \overline{B_i} = 1$ for every bit, so Y is all 1s, which is -1 in 2's complement. $F = A + (-1) + 0 = A - 1$: decrement A .

2 Computer arithmetic and floating point

Lecture: Ch3 slides 3–22 (ART Sir); Assignment 2 (half-precision adder) • **Textbook:** COD 5e §3.2, §3.3, §3.5

2.1 Integer addition, subtraction and overflow

Lecture: Ch3 slides 3–4 • **Textbook:** COD §3.2, Fig 3.1 (binary addition, $7 + 6$)

Subtraction is addition of the negation: $7 - 6 = 7 + (-6)$, where -6 is the 2's complement of 6. Overflow happens when the true result does not fit:

Operation	Overflow when
positive + negative	never
positive + positive	the result's sign bit is 1
negative + negative	the result's sign bit is 0
positive – negative	the result's sign bit is 1
negative – positive	the result's sign bit is 0
same signs subtracted	never

2.2 Multiplication in one paragraph

Lecture: Ch3 slides 5–8 • **Textbook:** COD §3.3

The lecture's long-multiplication hardware tests the multiplier's lowest bit; if it is 1 it adds the multiplicand into the product, then shifts the multiplicand left and the multiplier right, 32 times. The optimised version shifts the product register right instead, so a 32-bit ALU suffices. A product needs as many bits as both operands together, which is why MIPS keeps it in the two registers HI and LO (`mult/multu`); `mul rd, rs, rt` keeps only the low 32 bits.

2.3 Floating-point representation

Lecture: Ch3 slides 9–11 • **Textbook:** COD §3.5

A floating-point number is binary scientific notation: $\pm 1.xxxx_2 \times 2^{yyyy}$. *Normalised* means exactly one non-zero digit before the binary point, and in binary that digit is always 1.

$$x = (-1)^S \times (1 + \text{Fraction}) \times 2^{(\text{Exponent} - \text{Bias})}$$

Format	Sign	Exponent	Fraction	Bias	Total
Half (Assignment 2)	1	5	10	15	16
Single	1	8	23	127	32
Double	1	11	52	1023	64

- **Hidden bit.** The leading 1 of a normalised significand is not stored, so a 23-bit fraction gives 24 bits of precision.
- **Biased exponent.** The stored exponent is actual + Bias, with $\text{Bias} = 2^{k-1} - 1$ for k exponent bits. This keeps the field unsigned, so two floating-point numbers can be compared with an ordinary integer comparator (sign first, then exponent, then fraction) — no special hardware.
- **Reserved exponents.** All 0s and all 1s are reserved for special values (below). Normal exponents run from 1 to $2^k - 2$.

2.4 Range and precision

Lecture: Ch3 slides 12–14 • Textbook: COD §3.5

For any format with k exponent bits and m fraction bits

Bias = $2^{k-1} - 1$, smallest normalised = $1.0 \times 2^{1-\text{Bias}}$, largest = $(2 - 2^{-m}) \times 2^{(2^k-2)-\text{Bias}}$
 Smallest denormal (if denormals are allowed) = $2^{-m} \times 2^{1-\text{Bias}}$.

	Smallest normalised	Largest	Precision
Single	$1.0 \times 2^{-126} \approx 1.2 \times 10^{-38}$	$\approx 2.0 \times 2^{127} \approx 3.4 \times 10^{38}$	2^{-23} , about 6 decimal digits
Double	$1.0 \times 2^{-1022} \approx 2.2 \times 10^{-308}$	$\approx 2.0 \times 2^{1023} \approx 1.8 \times 10^{308}$	2^{-52} , about 16 decimal digits
Half	$2^{-14} \approx 6.1 \times 10^{-5}$	$(2 - 2^{-10}) \times 2^{15} = 65504$	2^{-10} , about 3 decimal digits

Decimal digits of precision $\approx m \times \log_{10} 2 \approx 0.3m$: $23 \times 0.3 \approx 6$, $52 \times 0.3 \approx 16$.

2.5 Special values

Lecture: Ch3 slides 17–18 • Textbook: COD §3.5, Fig 3.13 (IEEE 754 encoding of floating-point numbers)

Exponent	Fraction	Meaning
all 0s	0	± 0 — two zeros, +0 and -0
all 0s	$\neq 0$	denormal: $(-1)^S(0 + \text{Fraction}) \times 2^{1-\text{Bias}}$ — gradual underflow
1 to $2^k - 2$	anything	normalised number
all 1s	0	$\pm \infty$ — usable in later calculations, so no overflow check is needed
all 1s	$\neq 0$	NaN (Not a Number) — an illegal or undefined result such as 0/0

Denormals have no hidden 1, so they fill the gap between 0 and the smallest normalised number, with decreasing precision.

2.6 Converting

Lecture: Ch3 slides 15–16 • Textbook: COD §3.5 (the -0.75 and -5.0 examples)

Decimal to floating point. (1) Sign bit. (2) Write $|x|$ in binary. (3) Normalise to $1.f \times 2^e$. (4) Exponent field = $e + \text{Bias}$. (5) Fraction field = f , padded with zeros on the right.

Fast binary fractions

Write the fraction as an integer over a power of 2: $0.4375 = \frac{7}{16} = (111)_2 \times 2^{-4} = 1.11_2 \times 2^{-2}$, $0.28125 = \frac{9}{32} = (1001)_2 \times 2^{-5} = 1.001_2 \times 2^{-2}$. For fractions that are not exact (like 0.1), multiply by 2 repeatedly and take the integer part each time.

The lecture's two examples. $-0.75 = (-1)^1 \times 1.1_2 \times 2^{-1}$: $S = 1$, exponent = $-1 + 127 = 126 = 01111110$, fraction = $1000\dots 0 \Rightarrow 1\ 01111110\ 1000\dots 0$. Going back: $1\ 10000001\ 01000\dots 0$ has $S = 1$, exponent 129 $\Rightarrow e = 2$, significand 1.25, so $x = -1.25 \times 2^2 = -5.0$.

2.7 Floating-point addition

Lecture: Ch3 slides 19–22; Assignment 2 • **Textbook:** COD §3.5, Fig 3.14 (addition algorithm), Fig 3.15 (FP adder block diagram)

The four steps

1. **Align:** shift the significand with the *smaller* exponent right until the exponents match.
2. **Add** the significands (subtract if the signs differ).
3. **Normalise** the result and check for overflow/underflow.
4. **Round** to the available fraction bits; renormalise if rounding carried out.

Decimal example (4 digits): $9.999 \times 10^1 + 1.610 \times 10^{-1}$. Align: $9.999 \times 10^1 + 0.016 \times 10^1$. Add: 10.015×10^1 . Normalise: 1.0015×10^2 . Round to 4 digits: 1.002×10^2 .

Binary example (4 bits): $0.5 + (-0.4375) = 1.000_2 \times 2^{-1} + (-1.110_2 \times 2^{-2})$. Align: $-1.110 \times 2^{-2} \rightarrow -0.111 \times 2^{-1}$. Add: $1.000 - 0.111 = 0.001_2 \times 2^{-1}$. Normalise: $1.000_2 \times 2^{-4}$, no over/underflow. Round: no change. Result = 0.0625.

The hardware (lecture's FP adder figure, and your Assignment 2) mirrors the steps: a *small ALU* subtracts the exponents and the control uses the difference to steer multiplexers; a *shifter* moves the smaller significand right; the *big ALU* adds; then a *normaliser* (shift left or right, with an increment/decrement of the exponent) and the *rounding hardware*, which can loop back to normalise again. It is much slower than an integer add, so real FP adders take several cycles and are pipelined.

Guard, round and sticky

Alignment shifts bits off the right-hand end. Keeping two extra bits (guard and round) plus a sticky bit (the OR of everything shifted out) lets the rounding step reproduce the correctly rounded result. The IEEE default rounding mode is round-to-nearest, ties to even.

2.8 Practice

Practice 2.1. Represent 9.75 in half precision.

Solution. $9.75 = 1001.11_2 = 1.00111_2 \times 2^3$. $S = 0$; exponent = $3 + 15 = 18 = 10010$; fraction = 0011100000 . Word = $0\ 10010\ 0011100000 = 0x48E0$.

Practice 2.2. Represent -12.375 in half precision, then decode it back.

Solution. $12.375 = 1100.011_2 = 1.100011_2 \times 2^3$. $S = 1$; exponent = $18 = 10010$; fraction = 1000110000 . Word = $1\ 10010\ 1000110000$. Grouped in fours: $1100\ 1010\ 0011\ 0000 = 0xCA30$. Back: $-(1 + 0.5 + 0.03125 + 0.015625) \times 2^3 = -1.546875 \times 8 = -12.375$. *Grouping trap:* write the 16 bits out in full before splitting into hex digits — splitting the fields (1, 5, 10) directly into hex gives a wrong answer.

Practice 2.3. What does the half-precision word $0xC8C0$ represent?

Solution. $0xC8C0 = 1100\ 1000\ 1100\ 0000 = 1\ 10010\ 0011000000$: $S = 1$, exponent $18 \Rightarrow e = 3$, fraction $.0011_2 = 0.1875$. Value = $-1.1875 \times 8 = -9.5$.

Practice 2.4. Represent 0.1 in single precision. Why is it not exact?

Solution. $0.1 = 0.\overline{00011}_2 = 1.1001\ 1001\ 1001 \dots_2 \times 2^{-4}$. Exponent = $-4 + 127 = 123 = 01111011$. The fraction repeats forever; rounded to 23 bits it is 10011001100110011001101 (the last bit rounds up). Word = $0x3DCCCCD$. It is not exact because $1/10$ has a factor of 5 in its denominator, and only fractions whose denominators are powers of 2 terminate in binary.

Practice 2.5. An 8-bit “mini-float” uses 1 sign, 3 exponent and 4 fraction bits. Give the bias, the largest number and the smallest positive normalised number.

Solution. Bias = $2^2 - 1 = 3$. Largest: exponent field 110 $\Rightarrow e = 6 - 3 = 3$, value = $(2 - 2^{-4}) \times 2^3 = 15.5$. Smallest normalised: exponent field 001 $\Rightarrow e = -2$, value = $2^{-2} = 0.25$.

Practice 2.6. A 16-bit format has 1 sign, 6 exponent and 9 fraction bits. Give the bias and the range.

Solution. Bias = $2^5 - 1 = 31$. Largest = $(2 - 2^{-9}) \times 2^{62-31} = (2 - 2^{-9}) \times 2^{31} \approx 4.29 \times 10^9$. Smallest normalised = $2^{1-31} = 2^{-30} \approx 9.31 \times 10^{-10}$.

Practice 2.7. Add 1.75 and -0.28125 , showing all four steps.

Solution. $1.75 = 1.11_2 \times 2^0$; $0.28125 = 1.001_2 \times 2^{-2}$. Align: $1.001 \times 2^{-2} \rightarrow 0.01001 \times 2^0$. Subtract: $1.11000 - 0.01001 = 1.00111_2 \times 2^0$. Already normalised; nothing to round. Result = $1.00111_2 = 1.46875$.

Practice 2.8. Add the half-precision numbers 0x4A00 and 0x4100.

Solution. 0x4A00 = 0 10010 1000000000 = $1.5 \times 2^3 = 12$; 0x4100 = 0 10000 0100000000 = $1.25 \times 2^1 = 2.5$. Align: $1.25 \times 2^1 = 0.3125 \times 2^3$. Add: $1.5 + 0.3125 = 1.8125$, so $1.8125 \times 2^3 = 14.5$. Encode: exponent 18 = 10010, $1.8125 = 1.1101_2 \Rightarrow$ fraction 1101000000. Word = 0 10010 1101000000 = 0x4B40.

Practice 2.9. Classify: 0x7F800000, 0xFF800001, 0x80000000, 0x00000001 (single precision).

Solution. $+\infty$ (exponent all 1s, fraction 0); NaN (exponent all 1s, fraction $\neq 0$); -0 (sign 1, all else 0); the smallest denormal, $2^{-23} \times 2^{-126} = 2^{-149} \approx 1.4 \times 10^{-45}$.

Practice 2.10. Why does IEEE 754 have two zeros, and is that a problem?

Solution. Zero is encoded as exponent 0 and fraction 0, and the sign bit is independent of both, so $+0$ and -0 both exist. They compare equal. The sign is kept because it carries information — for example $1/(+0) = +\infty$ but $1/(-0) = -\infty$.

Practice 2.11. Why can FP addition not be done in one short clock cycle?

Solution. It is a chain of dependent steps — exponent compare, alignment shift, add, leading-zero count, normalising shift, round, possibly normalise again. Doing all of them in one cycle would stretch the clock for every other instruction, so FP adders are multi-cycle and pipelined.

3 The MIPS instruction set and assembly

Lecture: Ch2 slides 3–55 (ART Sir); Assignment 4 (MIPS programming) • **Textbook:** COD 5e Chapter 2, §2.2–2.10

3.1 Registers and the calling convention

Lecture: Ch2 slides 6, 36–37 • **Textbook:** COD §2.8, Fig 2.11 (what is preserved across a call), Fig 2.14 (register conventions)

Register	Number	Role	Preserved across a call?
\$zero	0	constant 0	—
\$v0–\$v1	2–3	return values	no
\$a0–\$a3	4–7	arguments	no
\$t0–\$t7	8–15	temporaries	no — the caller must save them if needed
\$s0–\$s7	16–23	saved values	yes — the callee must save them before use
\$t8–\$t9	24–25	more temporaries	no
\$sp	29	stack pointer	yes
\$ra	31	return address	yes (saved by any procedure that calls another)

3.2 The three instruction formats

Lecture: Ch2 slides 19–23 • **Textbook:** COD §2.5, Fig 2.5 (MIPS instruction encoding)

R	op (6)	rs (5)	rt (5)	rd (5)	shamt (5)	funct (6)
I	op (6)	rs (5)	rt (5)	immediate / offset (16)		
J	op (6)	target address (26)				

Design principles behind them: all instructions are 32 bits; the fields that mean the same thing are in the same place in every format (op always 31–26, rs always 25–21, rt always 20–16), so the hardware can read registers before it knows the format.

3.3 Opcodes to memorise

Lecture: Ch2 slides 20–23 • **Textbook:** COD Fig 2.5 and the MIPS reference data card (front of the book)

Instr	Type	op	funct	Instr	Type	op	funct
add	R	000000	100000	addi	I	001000	—
addu	R	000000	100001	addiu	I	001001	—
sub	R	000000	100010	andi	I	001100	—
subu	R	000000	100011	ori	I	001101	—
and	R	000000	100100	xori	I	001110	—
or	R	000000	100101	lui	I	001111	—
xor	R	000000	100110	lw	I	100011	—
nor	R	000000	100111	sw	I	101011	—
slt	R	000000	101010	lb	I	100000	—
sltu	R	000000	101011	lbu	I	100100	—
sll	R	000000	000000	sb	I	101000	—
srl	R	000000	000010	beq	I	000100	—
sra	R	000000	000011	bne	I	000101	—
jr	R	000000	001000	slti	I	001010	—
j	J	000010	—	jal	J	000011	—

Memory hooks

- Loads start 100, stores start 101: lw=100011, sw=101011.
- Branches are 0001xx: beq=000100, bne=000101. Jumps are 00001x: j=000010, jal=000011.
- Immediate ALU operations are 001xxx: addi 000, slti 010, andi 100, ori 101, lui 111.
- R-type functs for ALU operations are 10xxxx, in the same order: add 100000, sub 100010, and 100100, or 100101, nor 100111, slt 101010.

3.4 Addressing modes and target calculations

Lecture: Ch2 slides 51–55 (branch/jump addressing, “Branching Far Away”, addressing-mode summary) • **Textbook:** COD §2.10

Mode	Example	Operand / target
Register	add \$t0,\$t1,\$t2	registers only
Immediate	addi \$t0,\$t1,5	constant in the instruction, sign-extended to 32 bits
Base + displacement	lw \$t0,8(\$s0)	memory address = \$s0 + 8
PC-relative	beq \$t0,\$t1,L	target = $PC + 4 + (\text{offset} \times 4)$
Pseudo-direct	j L	target = $\{PC+4[31:28], \text{address} \times 4\}$

Branch offset = $(\text{target} - (PC + 4))/4$, a signed 16-bit count of *instructions*, so a branch reaches about $\pm 32K$ instructions. **Jump field** = target/4 with the top 4 bits dropped, so a jump reaches anywhere in the current 256 MB region. If a conditional branch must go further, invert the condition and branch over an unconditional j.

3.5 Translating C

Lecture: Ch2 slides 29–35 (decisions and loops), 46–49 (bytes and strings) • **Textbook:** COD §2.7, §2.9

C	MIPS
if (a < b)	slt \$t0,\$a,\$b ; beq \$t0,\$zero,Else
if (a > b)	slt \$t0,\$b,\$a — swap the operands
if (a <= b)	slt \$t0,\$b,\$a ; bne \$t0,\$zero,Else — branch on the negation
while (c) {}	test at the top, j back at the bottom
do {} while (c)	body first, test at the bottom
A[i], int	sll \$t0,\$i,2 ; add \$t0,\$t0,\$A ; lw \$t1,0(\$t0)
A[i], char	no scaling; use lbu/sb
x » 1	sra for signed int, srl for unsigned
c1 && c2	two chained branches, either one exits

3.6 Procedures and the stack

Lecture: Ch2 slides 36–45 • Textbook: COD §2.8, Figs 2.11, 2.12 (stack allocation), 2.14

jal L saves $PC + 4$ in \$ra and jumps; jr \$ra returns. The stack grows toward lower addresses, and \$sp points at the last word in use.

The stack-frame recipe

1. List what must survive each jal: \$ra (if this procedure calls another), every \$s register you modify, and every value you still need *after* the call.
2. Prologue: addi \$sp,\$sp,-4k, then sw each value at 0(\$sp), 4(\$sp), ...
3. Body: arguments in \$a0–\$a3, result in \$v0.
4. Epilogue: lw them back, addi \$sp,\$sp,4k, jr \$ra.
5. A leaf procedure (calls nobody) that uses only \$t registers needs no frame at all.

Traps

Word arrays scale the index by 4, char arrays do not. slt only tests “less than”. Every \$t, \$a and \$v register is destroyed across a jal — save it or reload it. The frame offsets in the prologue and epilogue must match, and \$sp must be restored before jr \$ra.

3.7 Practice

Practice 3.1. Encode add \$t0, \$s1, \$s2 in binary and hex.

Solution. op=000000, rs=\$s1=17=10001, rt=\$s2=18=10010, rd=\$t0=8=01000, shamt=00000, funct=100000. 000000 10001 10010 01000 00000 100000 = 0x02324020.

Practice 3.2. Encode lw \$t1, 8(\$s3).

Solution. op=100011, rs=\$s3=19=10011, rt=\$t1=9=01001, offset=8. 100011 10011 01001 0000000000001000 = 0x8E690008.

Practice 3.3. Decode 0x2129FFFF.

Solution. 001000 01001 01001 1111111111111111: op=addi, rs=rt=9=\$t1, immediate = -1. So addi \$t1, \$t1, -1.

Practice 3.4. A beq at 0x00400010 must reach 0x00400030. What is the 16-bit offset?

Solution. (0x00400030 - 0x00400014)/4 = 0x1C/4 = 7. Field = 0000000000000111.

Practice 3.5. A j at 0x00400020 must reach 0x00400100. What is the 26-bit field?

Solution. Both addresses share the top 4 bits (0), so the jump is legal. Field = $0x00400100/4 = 0x0100040$.

Practice 3.6. Translate `int sum=0; for(i=0;i<n;i++) sum += A[i];` with `A=$a0, n=$a1, i=$t0`, result in `$v0`.

Solution.

```

    add $v0, $zero, $zero      # sum = 0
    add $t0, $zero, $zero      # i = 0
Loop: slt $t1, $t0, $a1        # i < n ?
    beq $t1, $zero, Exit
    sll $t2, $t0, 2            # i*4
    add $t2, $t2, $a0          # EA[i]
    lw  $t3, 0($t2)
    add $v0, $v0, $t3          # sum += A[i]
    addi $t0, $t0, 1
    j   Loop
Exit: jr  $ra

```

A leaf procedure using only `$t` registers — no stack frame needed.

Practice 3.7. Write a recursive `fact(n)` in MIPS.

Solution.

```

fact: addi $sp, $sp, -8
    sw  $ra, 4($sp)
    sw  $a0, 0($sp)          # n is needed after the call
    slti $t0, $a0, 1         # n < 1 ?
    beq $t0, $zero, Rec
    addi $v0, $zero, 1        # base: return 1
    addi $sp, $sp, 8
    jr  $ra
Rec:  addi $a0, $a0, -1
    jal fact                 # v0 = fact(n-1)
    lw  $a0, 0($sp)          # restore n
    lw  $ra, 4($sp)
    addi $sp, $sp, 8
    mul $v0, $v0, $a0        # n * fact(n-1)
    jr  $ra

```

Practice 3.8. A procedure `f` calls `g` inside a loop and uses a loop counter, a running total and the array base after each call. What goes on the stack?

Solution. `$ra` (because `f` calls `g`) and the `$s` registers holding the counter, total and base — those values must live in `$s` registers precisely because they survive the calls, and `f` must save the `$s` registers it overwrites. Nothing else.

Practice 3.9. Count the characters equal to `c` in a null-terminated string: `s` in `$a0`, `c` in `$a1`, result in `$v0`.

Solution.

```

    add $v0, $zero, $zero      # n = 0
Loop: lbu $t0, 0($a0)          # *s (byte!)
    beq $t0, $zero, Exit       # '\0'
    bne $t0, $a1, Skip
    addi $v0, $v0, 1
Skip: addi $a0, $a0, 1          # s++ : one byte, not four

```

```

        j      Loop
Exit: jr      $ra

```

4 The single-cycle datapath and control

Lecture: Ch4 Part 1 slides 12–33 (RS Sir); control-unit notes; Assignment 3 • **Textbook:** COD 5e §4.3–4.4 and Appendix D

4.1 The building blocks

Lecture: Ch4 Part 1 slides 12–22 • **Textbook:** COD §4.3, Fig 4.2 (basic implementation with multiplexors and control lines)

Element	Role
PC	32-bit register holding the address of the current instruction
Instruction memory	read-only during execution; outputs the instruction at the PC
Adder ($PC + 4$)	computes the next sequential address
Register file	32 registers, two read ports (rs, rt) and one write port (controlled by RegWrite)
Sign-extend	widens the 16-bit immediate to 32 bits
ALU	add, subtract, AND, OR, slt; outputs a result and a Zero flag
Data memory	read (MemRead) or written (MemWrite) at the ALU's address
Shift left 2 + adder	computes the branch target $PC + 4 + (SE(imm) \ll 2)$
Four multiplexers	RegDst, ALUSrc, MemtoReg, PCSrc — labelled m2 , m3 , m4 , m1 in exam figures

Two memories are needed because a single-cycle machine fetches an instruction and accesses data in the same cycle. Figure 1 shows the complete datapath as printed on exam papers.

4.2 The control signals

Lecture: Ch4 Part 1 slides 23–27 • **Textbook:** COD §4.4, Fig 4.13 (ALU control truth table), Fig 4.22 (main control truth table)

Signal	When 0	When 1
RegDst (m2)	write register = rt (bits 20–16)	write register = rd (bits 15–11)
ALUSrc (m3)	ALU operand B = Read data 2	ALU operand B = sign-extended immediate
MemtoReg (m4)	write-back value = ALU result	write-back value = memory data
PCSrc (m1)	next PC = $PC + 4$	next PC = branch target
RegWrite	nothing	register file written
MemRead	nothing	data memory read
MemWrite	nothing	data memory written
Branch	—	PCSrc = Branch AND Zero
ALUOp (2 bits)	00 add, 01 subtract, 10 look at the funct field	

Instruction	RegDst	ALUSrc	MemtoReg	RegWrite	MemRead	MemWrite	Branch	ALUOp
R-format	1	0	0	1	0	0	0	10
lw	0	1	1	1	1	0	0	00
sw	X	1	X	0	0	1	0	00
beq	X	0	X	0	0	0	1	01

Table 5: Main control truth table. COD Fig 4.22 = control-unit notes Fig D.2.4; Ch4 Part 1 slide 25. X = don't care.

ALUOp	Instruction	funct	ALU control
00	lw, sw	xxxxxx	0010 add
01	beq	xxxxxx	0110 subtract
10	R-type	100000	0010 add
10	R-type	100010	0110 subtract
10	R-type	100100	0000 AND
10	R-type	100101	0001 OR
10	R-type	101010	0111 set on less than

Table 6: ALU control. COD Fig 4.13 = control-unit notes Fig D.2.1; Ch4 Part 1 slides 23–24. Two levels keep the main control small: it only has to decide the ALUOp class.

Why the don't-cares. For **sw** and **beq**, **RegWrite** = 0, so nothing is written back: whatever **RegDst** and **MemtoReg** select is thrown away.

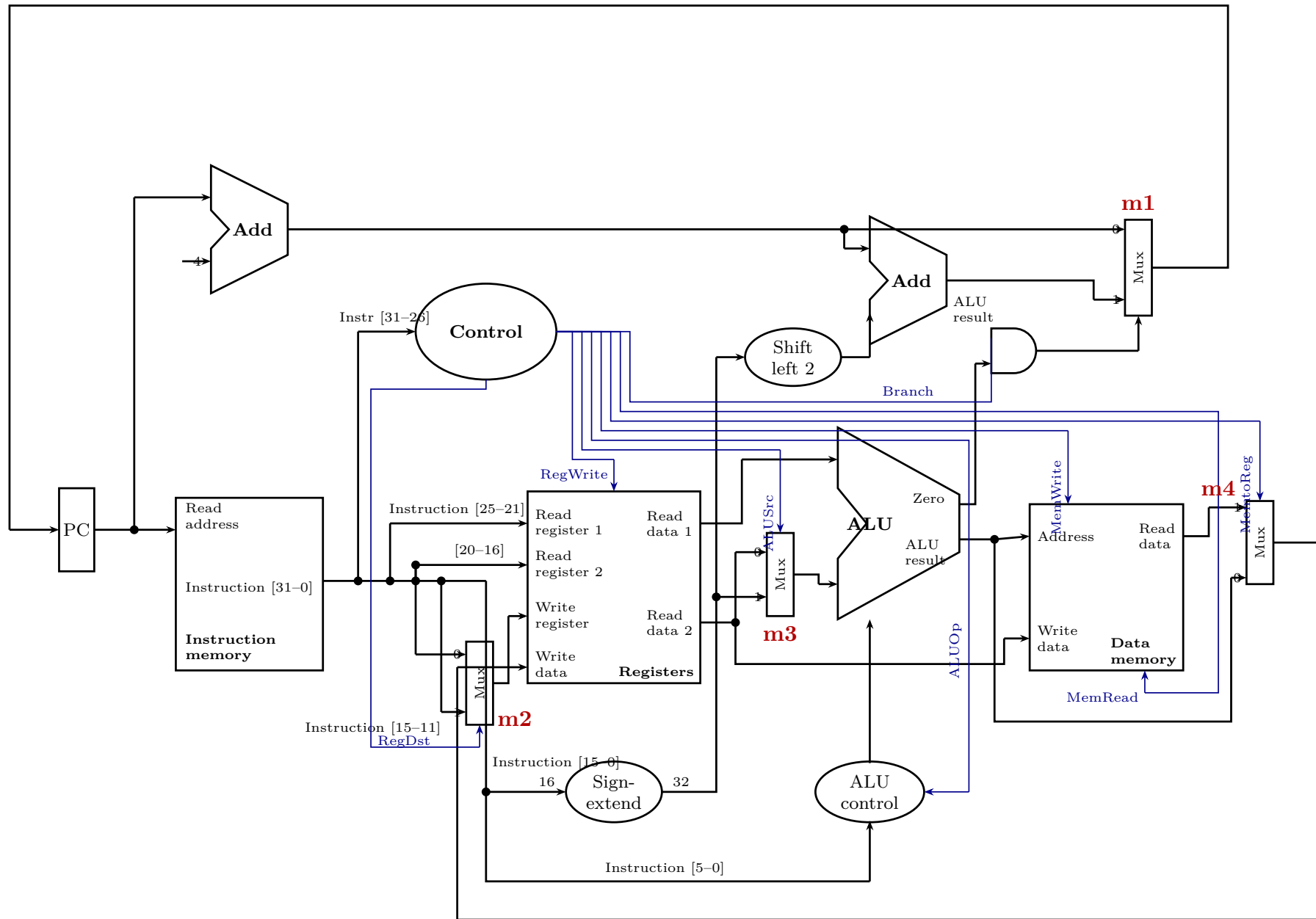


Figure 1: The single-cycle MIPS datapath with control (COD Fig 4.17; Ch4 Part 1 slide 26). Blue lines are control signals. **m1**=PCSrc, **m2**=RegDst, **m3**=ALUSrc, **m4**=MemtoReg.

4.3 How each instruction uses the datapath

Lecture: Ch4 Part 1 slides 28–32 • **Textbook:** COD §4.4, Figs 4.19 (R-type), 4.20 (load), 4.21 (branch-on-equal), 4.24 (jump)

R-type (`add $s1,$s2,$s3`): fetch; read rs and rt; ALU operates on the two register values ($ALUSrc = 0$); the result returns through m4 ($MemtoReg = 0$) to register rd ($RegDst = 1$); $PC \leftarrow PC + 4$.

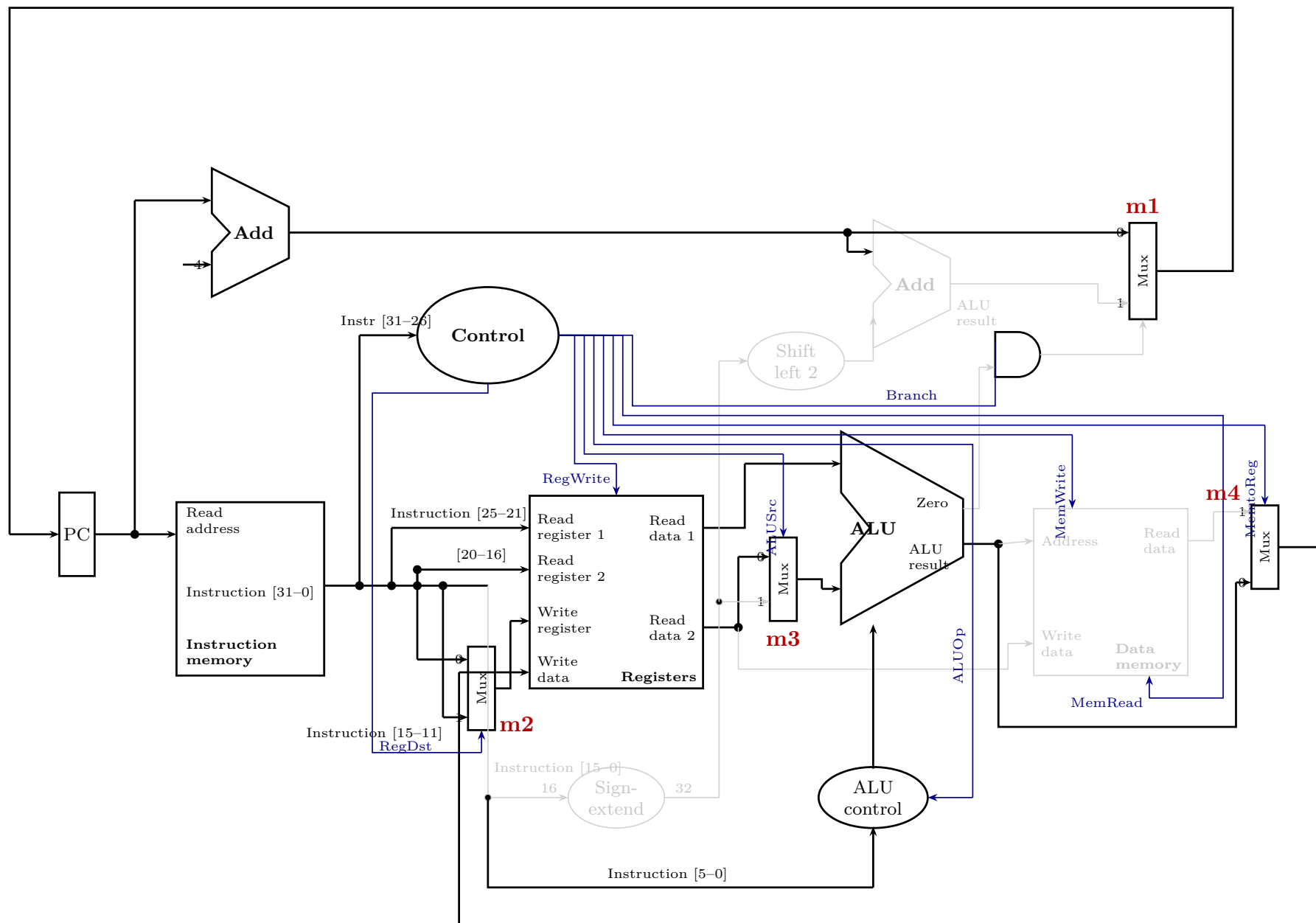


Figure 2: R-type (COD Fig 4.19; slide 28). RegDst=1, ALUSrc=0, MemtoReg=0, RegWrite=1, MemRead=0, MemWrite=0, Branch=0, ALUOp=10. **m1=0, m2=1, m3=0, m4=0**. Sign-extend, shift-left-2, the branch adder and the data memory are idle.

Load (`lw $t1,8($s3)`): read the base register `rs`; sign-extend the offset and add it in the ALU (`ALUSrc = 1`); read data memory at that address; write the data (`MemtoReg = 1`) into `rt` (`RegDst = 0`). This is the longest path in the machine, so it sets the clock period.

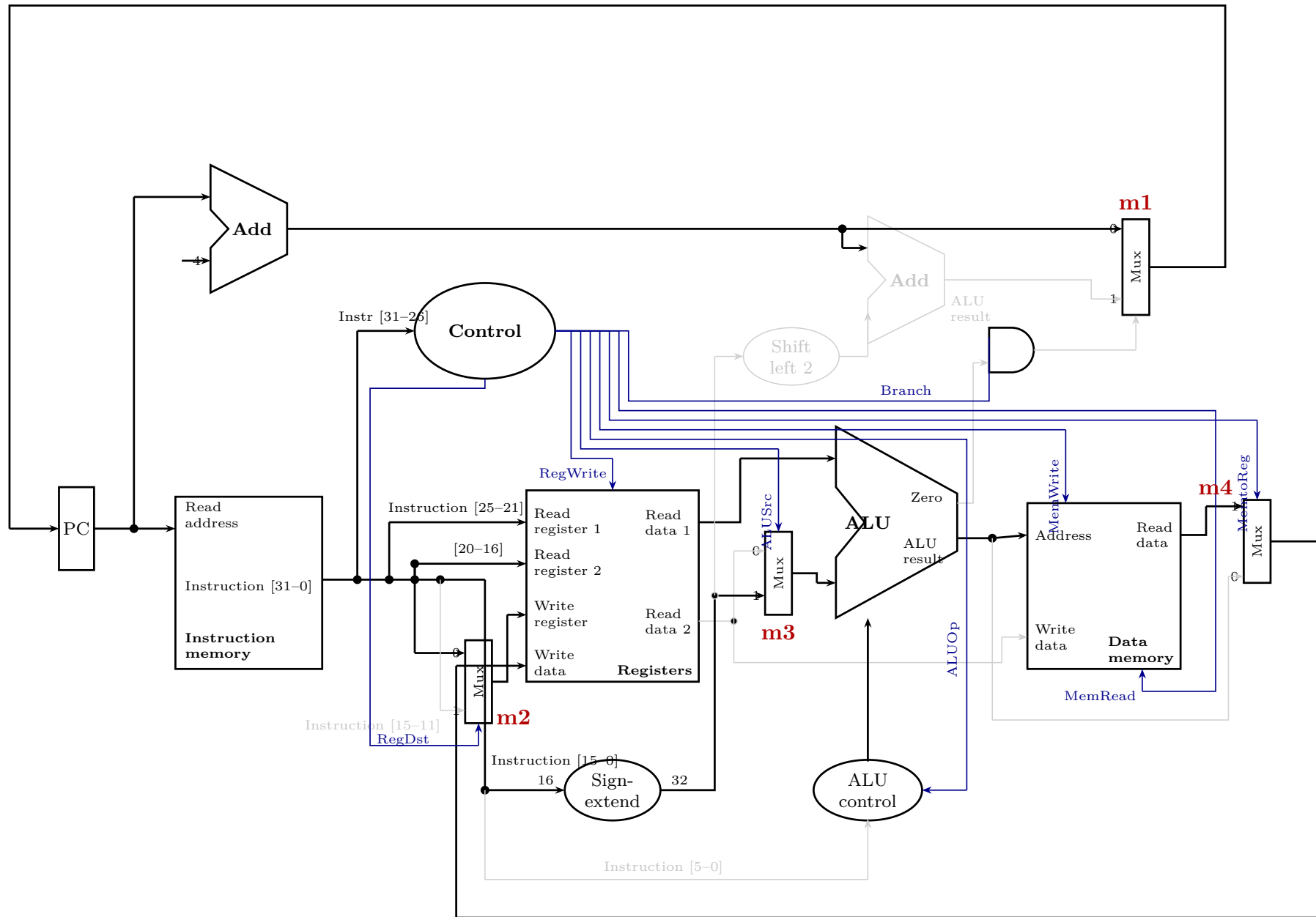


Figure 3: 1w (COD Fig 4.20; slide 29). $\text{RegDst}=0$, $\text{ALUSrc}=1$, $\text{MemtoReg}=1$, $\text{RegWrite}=1$, $\text{MemRead}=1$, $\text{MemWrite}=0$, $\text{Branch}=0$, $\text{ALUOp}=00$. **m1**=0, **m2**=0, **m3**=1, **m4**=1. Instruction bits 15–11 are unused.

Store (`sw $t0,-8($t1)` — your MIPS online question): the address is computed exactly as for `lw`; the offset `-8` is sign-extended to `0xFFFFFFFF8`. Read data 2 (the value of `rt`) goes to the memory's Write data port and `MemWrite = 1`. No register is written, so the whole write-back path is idle.

25

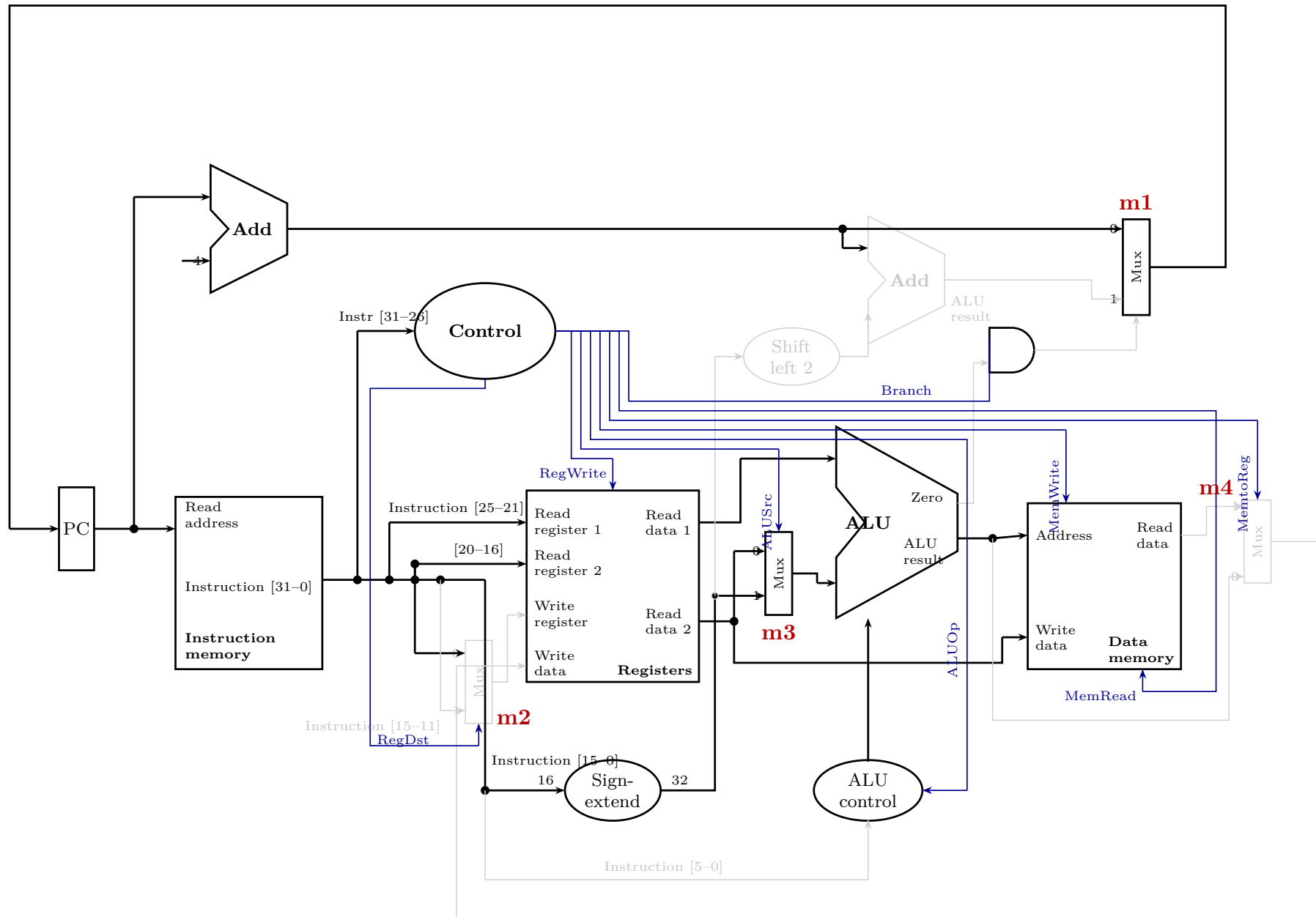


Figure 4: `sw $t0, -8($t1)` (not drawn in COD or the slides; derived from the load figure). `RegDst=X`, `ALUSrc=1`, `MemtoReg=X`, `RegWrite=0`, `MemRead=0`, `MemWrite=1`, `Branch=0`, `ALUOp=00`. **m1=0**, **m2=X**, **m3=1**, **m4=X**.

Branch (`beq $s1,$s2,L`): read both registers; the ALU subtracts ($ALUOp = 01$) and raises Zero if they are equal; in parallel the offset is sign-extended, shifted left 2 and added to $PC + 4$; the AND gate makes $PCSrc = Branch \cdot Zero$. PCSrc is the one mux select that depends on data rather than the opcode.

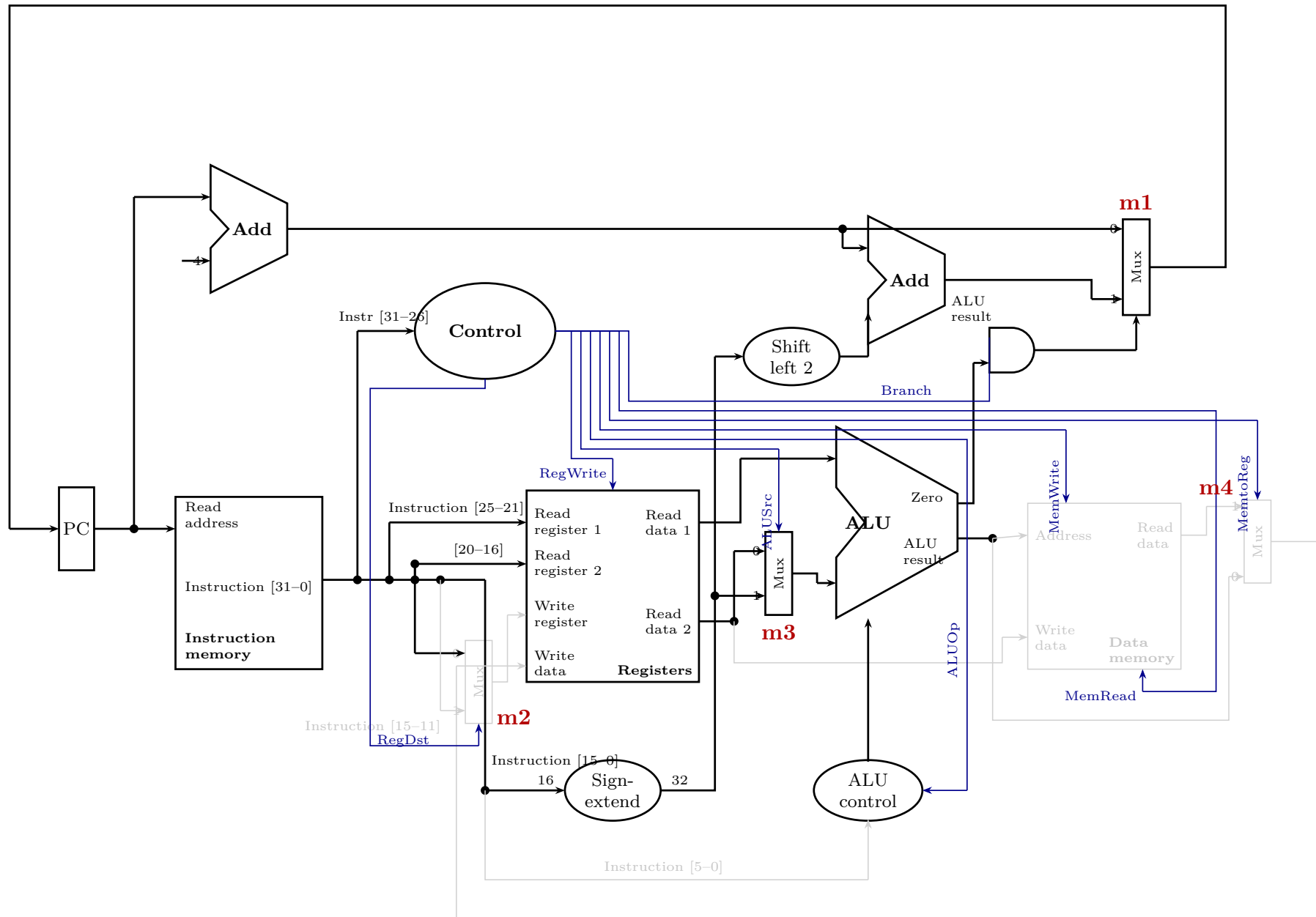


Figure 5: `beq` (COD Fig 4.21; slide 30). `RegDst=X`, `ALUSrc=0`, `MemtoReg=X`, `RegWrite=0`, `MemRead=0`, `MemWrite=0`, `Branch=1`, `ALUOp=01`. **m1**=1 if taken, 0 if not; **m2**=X, **m3**=0, **m4**=X.

Jump (j L) needs hardware the basic datapath lacks: a shift-left-2 of instruction bits 25–0 (26 bits become 28), concatenation with $PC+4[31:28]$, a third multiplexer in front of the PC, and a **Jump** control signal. The jump mux must come after the branch mux so that a jump overrides it.

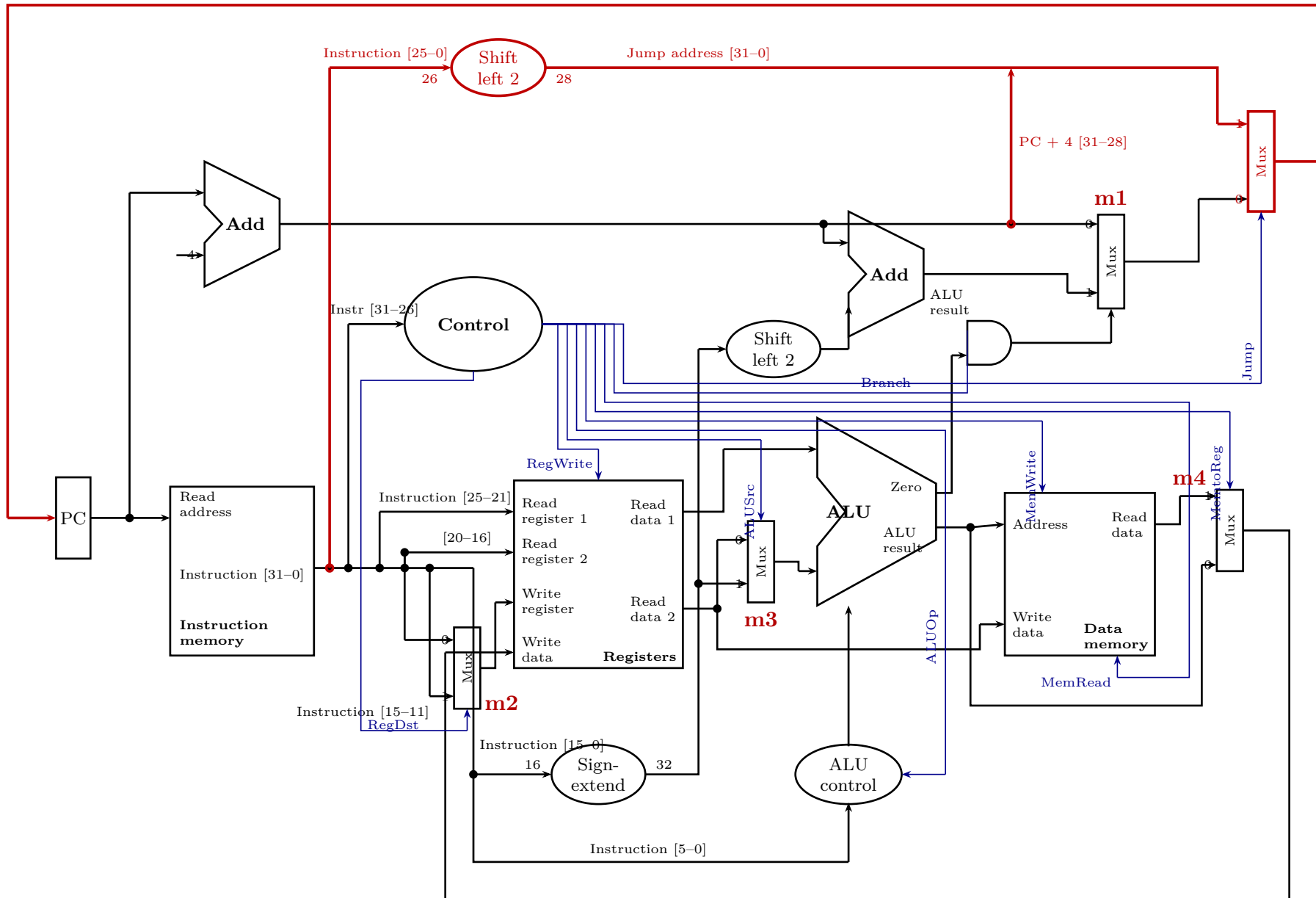


Figure 6: Datapath with jumps added, the new hardware in red (COD Fig 4.24; Ch4 Part 1 slides 31–32).

4.4 Implementing the control unit

Lecture: control-unit notes, Figs D.2.1–D.2.5 (combinational), D.3.1 (FSM) • **Textbook:** COD Appendix D, §D.2–D.3

Each instruction class is one AND term of the opcode bits:

$$\begin{aligned} R &= \overline{Op_5} \overline{Op_4} \overline{Op_3} \overline{Op_2} \overline{Op_1} \overline{Op_0} & (000000) & \quad 1w = Op_5 \overline{Op_4} \overline{Op_3} \overline{Op_2} Op_1 Op_0 & (100011) \\ sw &= Op_5 \overline{Op_4} Op_3 \overline{Op_2} Op_1 Op_0 & (101011) & \quad beq = \overline{Op_5} \overline{Op_4} \overline{Op_3} Op_2 \overline{Op_1} \overline{Op_0} & (000100) \end{aligned}$$

and each control signal is an OR of those terms:

$$\begin{aligned} \text{RegDst} &= R & \text{ALUSrc} &= 1w + sw & \text{MemtoReg} &= 1w \\ \text{RegWrite} &= R + 1w & \text{MemRead} &= 1w & \text{MemWrite} &= sw \\ \text{Branch} &= beq & \text{ALUOp1} &= R & \text{ALUOp0} &= beq \end{aligned}$$

This two-level AND–OR structure is a **PLA** (Figure D.2.5). The alternative is a **ROM**: the 6 opcode bits address $2^6 = 64$ words of 9 control bits each (64×9). A ROM stores every input combination; a PLA stores only the product terms that are needed, so it is smaller when most combinations are unused.

Multicycle (FSM) control, in one table — useful for short questions:

State	Step	Main actions
0	instruction fetch	MemRead, IRWrite, $PC \leftarrow PC + 4$
1	decode / register fetch	read registers, precompute branch target
2	memory address (1w/sw)	ALU adds base + offset
3 / 5	memory access	read (1w) / write (sw)
4	write-back (1w)	RegWrite, MemtoReg=1
6 / 7	R-type execute / completion	ALU operates; then RegWrite, RegDst=1
8	branch completion	compare, PCWriteCond
9	jump completion	PCWrite, PCSource=10

4.5 Practice

Practice 4.1. Fill the control table for add, lw, sw, beq, addi, slt.

	RegDst	ALUSrc	MemtoReg	RegWrite	MemRead	MemWrite	Branch	ALUOp
add								
lw								
sw								
beq								
addi								
slt								

Solution. add 1,0,0,1,0,0,0,10 lw 0,1,1,1,1,0,0,00 sw X,1,X,0,0,1,0,00 beq X,0,X,0,0,0,1,01
 addi 0,1,0,1,0,0,0,00 (destination rt, operand the immediate, ALU adds) slt 1,0,0,1,0,0,0,10 (an ordinary R-type; the ALU control decodes funct 101010 to 0111).

Practice 4.2. Give m1–m4 for add and for lw.

Solution. add: m1=0, m2=1, m3=0, m4=0. lw: m1=0, m2=0, m3=1, m4=1.

Practice 4.3. Which units are idle during beq?

Solution. The data memory, the MemtoReg mux and the whole write-back path, and the RegDst mux. (The register file is read but not written.)

Practice 4.4. Why is PCSrc not an output of the control unit?

Solution. Because it depends on data: $PCSrc = \text{Branch} \cdot \text{Zero}$, and Zero is known only after the ALU compares the registers. The control unit sees only the opcode.

Practice 4.5. With $ALUOp = 10$ and $funct = 100101$, what does the ALU do?

Solution. or: the ALU control outputs 0001.

Practice 4.6. How big is a ROM implementing the single-cycle main control? Why is a PLA smaller?

Solution. $2^6 = 64$ words $\times$ 9 bits = 576 bits. Only four opcodes matter, so the ROM is mostly unused entries; a PLA needs only four AND terms (one per instruction class) and nine OR terms.

Practice 4.7. Which instruction sets the clock period, and why is that a weakness of single-cycle design?

Solution. lw: instruction memory $\rightarrow$ register read $\rightarrow$ ALU $\rightarrow$ data memory $\rightarrow$ register write. Every instruction must fit in that same long cycle, even ones that need far less time — the motivation for multicycle and pipelined designs.

Practice 4.8. What must be added to support `jr $rs`?

Solution. Read data 1 (the value of rs) as one more input to the PC multiplexer chain, plus a control signal to select it. Nothing else changes: RegWrite and MemWrite are 0.

Practice 4.9. Does `addi` need any new hardware?

Solution. No. $ALUSrc = 1$, $RegDst = 0$, $RegWrite = 1$, $ALUOp = 00$ — every piece is already there. Only the control unit needs to recognise opcode 001000.

5 Pipelining

Lecture: Ch4 Part 2 slides 2–85 (RS Sir) • **Textbook:** COD 5e §4.5–4.8

5.1 The idea

Lecture: Ch4 Part 2 slides 2–9 (analogy, stages, performance), 33–35 (pipelined datapath) • **Textbook:** COD §4.5–4.6, Fig 4.35 (the pipelined datapath)

Pipelining overlaps instructions the way a laundry overlaps washing, drying and folding: each instruction still takes the same time, but a new one starts every cycle. MIPS has five stages:

IF instruction fetch $\rightarrow$ **ID** decode and register read $\rightarrow$ **EX** execute or address calculation $\rightarrow$ **MEM** data memory access $\rightarrow$ **WB** write back.

Pipeline registers between the stages (IF/ID, ID/EX, EX/MEM, MEM/WB) hold everything the later stages need, including the destination register number and the control signals.

Timing

With stage time t , N instructions take $(5 + N - 1)$ cycles = $(4 + N)$ cycles, plus one per stall. Example: if single-cycle needs 800 ps per instruction and the longest stage is 200 ps, three loads take $3 \times 800 = 2400$ ps unpipelined and $(4 + 3) \times 200 = 1400$ ps pipelined; for a long program the speedup approaches $800/200 = 4$. Ideal speedup equals the number of stages; real speedup is lower

because stages are unequal and hazards add stalls. Pipelining improves *throughput*, not the latency of one instruction.

5.2 Hazards

Lecture: Ch4 Part 2 slides 10–24 • Textbook: COD §4.5

Hazard	Cause	Remedy
Structural	two instructions need the same hardware in one cycle	separate instruction and data memories; write the register file in the first half-cycle and read it in the second
Data	an instruction needs a result that is not yet written back	forwarding (bypassing); a load followed immediately by a user still needs 1 stall
Control	the next instruction to fetch depends on a branch not yet resolved	stall; predict not-taken; resolve the branch in ID; delayed branch; dynamic prediction

5.3 Forwarding and stalls

Lecture: Ch4 Part 2 slides 53–76 (forwarding conditions 61–64, hazard detection 69–75) • Textbook: COD §4.7, Fig 4.56 (forwarding datapath), Fig 4.60 (hazard detection unit)

A result exists at the end of EX (ALU) or MEM (load), long before WB. Forwarding sends it straight to the ALU inputs through two extra multiplexers (ForwardA, ForwardB).

Forwarding conditions

EX hazard (the producer is one instruction ahead):

```
if (EX/MEM.RegWrite and EX/MEM.RegisterRd ≠ 0
    and EX/MEM.RegisterRd = ID/EX.RegisterRs) ForwardA = 10
```

MEM hazard (two ahead), only if the EX hazard does not already apply:

```
if (MEM/WB.RegWrite and MEM/WB.RegisterRd ≠ 0
    and not (EX hazard for Rs)
    and MEM/WB.RegisterRd = ID/EX.RegisterRs) ForwardA = 01
```

Replace Rs with Rt for ForwardB. The “not EX hazard” guard handles the *double data hazard*, where two earlier instructions both write the same register — the most recent one must win.

Load-use hazard. A load’s data appears only at the end of MEM, but the very next instruction wants it at the start of EX — backwards in time. The hazard detection unit in ID stalls for one cycle:

```
if (ID/EX.MemRead and (ID/EX.RegisterRt = IF/ID.RegisterRs or ID/EX.RegisterRt =
    IF/ID.RegisterRt)) stall
```

To stall: set PCWrite= 0 and IF/IDWrite= 0 (fetch and decode repeat) and force the control signals in ID/EX to 0 (a bubble, a “nop”, travels down the pipe).

5.4 Branch hazards

Lecture: Ch4 Part 2 slides 25–31, 77–85 • Textbook: COD §4.8, Fig 4.64 (scheduling the branch delay slot)

- **Predict not taken:** keep fetching sequentially; flush the wrong-path instructions if the branch is taken.
- **Resolve in ID:** move the comparator and branch adder into ID, so a taken branch costs 1 cycle instead of 3. The price: an operand produced by the instruction just before the branch must be forwarded into ID, which costs 1 stall for an ALU result (2 for a load).

- **Delayed branch:** always execute the instruction after the branch; the compiler fills that slot.
- **Dynamic prediction:** a branch history table. A 1-bit predictor mispredicts twice per loop visit (on the exit and on re-entry); a 2-bit predictor must be wrong twice before it flips, so it mispredicts only once.

5.5 Practice

Practice 5.1. How many stalls, and how many cycles? `lw $t0,0($s0); lw $t1,4($s0); add $t2,$t0,$t1; sw $t2,8($s0)`

Solution. One stall: the second `lw` feeds `add` immediately (load-use). The first `lw` is two ahead of `add`, so MEM/WB forwarding covers it; `sw` gets `$t2` by EX/MEM forwarding. Cycles = 4+4+1 = 9.

Practice 5.2. Same question: `add $t1,$t2,$t3; lw $t4,0($t1); sub $t5,$t4,$t1; sw $t5,4($t1)`

Solution. `add`→`lw` on `$t1`: EX/MEM forwarding, no stall. `lw`→`sub` on `$t4`: load-use, 1 stall. `sub`→`sw` on `$t5`: forwarding, no stall. Cycles = 4 + 4 + 1 = 9.

Practice 5.3. Draw the pipeline diagram for `add $t1,$t2,$t3` followed by `sub $t4,$t1,$t5`, and name the forwarding path.

Solution.

	CC1	CC2	CC3	CC4	CC5	CC6
add	IF	ID	EX	MEM	WB	
sub		IF	ID	EX	MEM	WB

`add`'s result sits in EX/MEM during CC4, exactly when `sub` is in EX: forward EX/MEM → ALU input A (ForwardA = 10). No stall.

Practice 5.4. Write the ForwardB condition for an EX hazard.

Solution. `if (EX/MEM.RegWrite and EX/MEM.RegisterRd ≠ 0 and EX/MEM.RegisterRd = ID/EX.RegisterRt) ForwardB = 10`

Practice 5.5. Why check that RegisterRd ≠ 0?

Solution. Register 0 is hard-wired to zero. An instruction that “writes” `$zero` must not have its result forwarded, or a later reader of `$zero` would get a non-zero value.

Practice 5.6. A loop branch is taken 9 times and then not taken, and the loop is entered repeatedly. How many mispredictions per visit with a 1-bit and with a 2-bit predictor?

Solution. 1-bit: two — the exit (predicted taken) and the first iteration of the next visit (predicted not taken). 2-bit: one — only the exit; a single miss is not enough to flip a strongly-taken state.

Practice 5.7. Why can forwarding never remove a load-use stall?

Solution. The data leaves memory at the end of MEM; the consumer needs it at the start of EX in the same clock cycle. No wire can send data backwards in time, so one bubble is unavoidable (unless the compiler reorders).

6 Performance

Lecture: Ch1 slides 14–27 (ART Sir) • **Textbook:** COD 5e §1.6 (performance), §1.10 (Amdahl's law and the MIPS pitfall)

The formulas

$$\text{CPU time} = \text{IC} \times \text{CPI} \times T_{clk} = \frac{\text{IC} \times \text{CPI}}{f}, \quad \text{CPI} = \sum_i \text{CPI}_i \times \frac{\text{IC}_i}{\text{IC}}$$

$$\text{Amdahl's law: } T_{new} = \frac{T_{affected}}{k} + T_{unaffected}, \quad \text{Speedup} = \frac{1}{(1-f) + f/k}$$

Performance is the product of three factors; improving one while worsening another can make things slower. Clock rate alone, or a MIPS rating across different instruction sets, is not a valid comparison. Amdahl's law says the part you do not improve limits the total speedup: with fraction f improved, the speedup can never exceed $1/(1-f)$.

6.1 Practice

Practice 6.1. Machine A: 1.5 GHz, CPI 1.2. Machine B: 2 GHz, CPI 1.8. Same ISA and program. Which is faster?

Solution. $T_A = \text{IC} \times 1.2/1.5\text{G} = 0.8 \text{ ns} \times \text{IC}$; $T_B = \text{IC} \times 1.8/2\text{G} = 0.9 \text{ ns} \times \text{IC}$. A is faster by $0.9/0.8 = 1.125\times$ despite B's higher clock.

Practice 6.2. A program has instruction classes A, B, C with CPI 1, 2, 3 and frequencies 50%, 30%, 20%. For 10^9 instructions at 2 GHz, find the CPI and the time.

Solution. $\text{CPI} = 0.5(1) + 0.3(2) + 0.2(3) = 1.7$. $\text{Time} = 10^9 \times 1.7/(2 \times 10^9) = 0.85 \text{ s}$.

Practice 6.3. A program runs 100 s, of which 80 s is multiplication. How much faster must multiplication be for the program to run $4\times$ faster? And $5\times$?

Solution. $4\times$: target 25 s; the other 20 s cannot shrink, so multiplication must take 5 s, a $16\times$ speedup. $5\times$: target 20 s, which would need multiplication to take 0 s — impossible. That is Amdahl's limit.

Practice 6.4. A design change cuts CPI by 20% but lowers the clock from 3 GHz to 2.5 GHz. Is it worth it?

Solution. $\text{Time} \propto \text{CPI}/f$: $0.8/2.5 = 0.32$ versus $1/3 = 0.333$. $\text{New} / \text{old} = 0.96$: about 4% faster, so marginally yes.

7 Cache (short)

Lecture: Ch5 slides 17–45 (RS Sir) • **Textbook:** COD 5e §5.3–5.4, Fig 5.15 (eight-block cache at each associativity)

A cache keeps recently used blocks of memory close to the processor, exploiting *temporal locality* (reuse soon) and *spatial locality* (neighbours soon). An address is split into three fields:

The address split

offset = $\log_2(\text{block size})$, index = $\log_2(\text{number of sets})$, tag = address bits – index – offset.
 Blocks = cache size / block size; sets = blocks / associativity (1 for direct-mapped; one set for fully associative).

Block size trade-off. Larger blocks exploit spatial locality and need fewer tags, but for a fixed cache size there are fewer blocks (more conflict misses), the miss penalty grows (more bytes to transfer), and unused bytes pollute the cache. Remedies: early restart / critical word first, wider or interleaved memory, prefetching.

7.1 Practice

Practice 7.1. A 16 KB direct-mapped cache with 32-byte blocks and 32-bit addresses: find blocks, and the tag, index and offset widths.

Solution. Blocks = $16384/32 = 512$; direct-mapped $\Rightarrow$ 512 sets. Offset 5 bits, index 9 bits, tag $32 - 9 - 5 = 18$ bits.

Practice 7.2. An 8 KB 2-way set-associative cache with 16-byte blocks and 32-bit addresses.

Solution. Blocks = $8192/16 = 512$; sets = $512/2 = 256$. Offset 4, index 8, tag $32 - 8 - 4 = 20$ bits.

Part II

Past papers, solved

Each paper is printed question by question, and each question is followed by its full solution. A grey line under each question points to the Part I section that teaches it and to the textbook source, so you can go back when a solution is not clear.

8 CSE 306 Quiz Examination (45 marks, 65 minutes)

About this paper

This is the scanned paper in your `quiz_questions` folder, set about three years ago when the sessional was numbered CSE 306. Its six questions: 9 MCQs (9), ALU input equations (12), IEEE 754 (6), 16-bit FP formats (6), pipelining (6), datapath marking (6).

Question 1 — MCQs ($9 \times 1 = 9$)

I. After an unsigned operation $A - B$, if you are shown only the value of the Carry flag and find it set (Carry Flag = 1), which of the following can you guarantee? (a) $A > B$ (b) $A < B$ (c) $A \not\geq B$ (d) $A \not\leq B$

Concept: §1, the arithmetic circuit • **Source:** Mano Table 9-2; ALU slide 11

Solution. (d) $A \not\leq B$. The ALU subtracts as $A + \overline{B} + 1$; a carry out means no borrow was needed, so $A \geq B$ (Table 9-2: “ $F = A - B$, $C_{out} = 1$ if $A \geq B$ ”). Not (a), because $A = B$ also gives $C = 1$.

II. Consider a 7-bit ALU. What is the function to determine the overflow (V) flag? C_1 is the first input carry bit. (a) $C_8 \oplus C_9$ (b) $C_8 + C_9$ (c) $C_8 C_9 + C'_8 C'_9$ (d) $C_7 C'_8 + C'_7 C_8$

Concept: §1, status flags • **Source:** Mano Fig 9-14; ALU slides 21–22

Solution. (d), which is $C_7 \oplus C_8$. V is the carry into the sign stage XOR the carry out of it. With C_1 entering stage 1, a 7-bit ALU has stages 1–7: the sign stage receives C_7 and emits C_8 . (For the lecture’s 8-bit ALU the same rule gives $C_8 \oplus C_9$ — option (a) is the trap.)

III. If $A = 0100$, $B = 0100$ and $C_{in} = 0$, which is FALSE after the add operation using a 4-bit ALU? (a) $Z = 0$ (b) $C = 0$ (c) $V = 0$ (d) $S = 1$

Concept: §1, status flags • **Source:** ALU slides 21–22; COD §3.2 (overflow)

Solution. (c). $0100 + 0100 = 1000$: $Z = 0$ (non-zero), $C = 0$ (no carry out), $S = 1$ (MSB). But $+4 + 4 = +8$ exceeds the 4-bit range (+7), and the carries are $C_4 = 1$ into the MSB, $C_5 = 0$ out, so $V = 1$. “ $V = 0$ ” is false.

IV. The formula for the bias of an arbitrarily sized floating-point number with k exponent bits is (a) $2^k - 1$ (b) $2^{(k-1)} - 1$ (c) 2^k (d) $2^{(k-1)}$

Concept: §2 • Source: COD §3.5

Solution. (b). Single precision: $k = 8 \Rightarrow 2^7 - 1 = 127$.

V. For IEEE 754, single precision (32-bit) and double precision (64-bit) numbers have x and y exponent bits. (a) 8 and 11 (b) 7 and 13 (c) 7 and 15 (d) 8 and 16

Concept: §2 • Source: COD §3.5; Ch3 slide 11

Solution. (a). $1 + 8 + 23 = 32$; $1 + 11 + 52 = 64$.

VI. What is NaN in IEEE standards? (a) Not a finite number (b) Not a node (c) Not a number (d) Not a normalized number

Concept: §2, special values • Source: COD §3.5, Fig 3.13; Ch3 slide 18

Solution. (c) Not a Number: exponent all 1s with a non-zero fraction, e.g. the result of $0/0$.

VII. Which MIPS instruction does NOT need any arithmetic operation? (a) `addi` (b) `beq` (c) `j` (d) none of the above

Concept: §4 • Source: COD §4.4, Fig 4.24

Solution. (c) j. `addi` adds and `beq` subtracts to compare, but `j` forms its target by shifting and concatenating bits only.

VIII. (The datapath-with-control figure is given.) Statement 1: for `beq`, `MemtoReg` can be 1. Statement 2: `addi` can be handled with the datapath. Statement 3: the only implementation difference between jump and branch is that the latter is conditional. Which is correct? (a) True 2, 3; False 1 (b) True 1, 3; False 2 (c) True 1, 2; False 3 (d) True 1; False 2, 3

Concept: §4 • Source: COD §4.4, Figs 4.17, 4.22, 4.24

Solution. (c). (1) True: `RegWrite = 0` for `beq`, so `MemtoReg` is a don't-care. (2) True: `addi` needs `ALUSrc = 1`, `RegDst = 0`, `RegWrite = 1`, `ALUOp = 00`, all already present. (3) False: the jump target is a 26-bit field shifted left 2 and concatenated with $PC+4[31:28]$, and it needs an extra multiplexer; a branch adds a sign-extended, shifted offset to $PC + 4$.

IX. How many instructions are in different stages in the same clock cycle in a 5-stage pipelined execution? (a) up to 4 (b) up to 5 (c) up to 6 (d) one

Concept: §5 • Source: COD §4.5

Solution. (b) up to 5, one per stage.

Question 2 — ALU input equations (12)

Question 2

Suppose $A(A_3A_2A_1A_0)$, $B(B_3B_2B_1B_0)$, S_2 , S_1 and $S_0(C_{in})$ are the inputs to your circuit, while X_i , Y_i and Z_i are the inputs to the i -th parallel adder. Derive the input equations (X_i , Y_i , Z_i) for the 4-bit parallel adders used in the ALU satisfying the specification below. Use as few gates as possible.

S_2	S_1	$S_0(C_{in})$	Required operation
0	0	0	Transfer A
0	0	1	2's complement of A
0	1	0	Add A with B
0	1	1	Add 2's complement of A with B
1	0	x	1's complement of A
1	1	x	A XOR B

Concept: §1, the derivation recipe • **Source:** Mano Tables 9-1, 9-3, 9-4; ALU slides 10–20

Solution. Step 1 — the adder inputs each row needs.

S_2	S_1	S_0	X_i	Y_i	carry	Reason
0	0	0	A_i	0	0	$F = A$
0	0	1	$\overline{A_i}$	0	1	$\overline{A} + 1$
0	1	0	A_i	B_i	0	$A + B$
0	1	1	$\overline{A_i}$	B_i	1	$\overline{A} + 1 + B$
1	0	x	$\overline{A_i}$	0	0	$F = \overline{A}$, carries killed
1	1	x	A_i	B_i	0	$F = A \oplus B$, carries killed

Step 2 — X_i . A_i is complemented in rows 001, 011 ($\overline{S_2}S_0$) and 10x ($S_2\overline{S_1}$):

$$X_i = A_i \oplus (\overline{S_2}S_0 + S_2\overline{S_1})$$

Step 3 — Y_i . $Y_i = B_i$ exactly in the rows with $S_1 = 1$, and 0 otherwise:

$$Y_i = S_1 B_i$$

Step 4 — Z_i . Arithmetic rows ($S_2 = 0$) ripple normally with the LSB carry-in equal to S_0 ; logic rows ($S_2 = 1$) must kill every carry:

$$Z_i = \overline{S_2} C_i, \quad C_1 = S_0 \Rightarrow Z_1 = \overline{S_2} S_0$$

Check ($A = 0110$, $B = 0011$): 000 $\rightarrow$ 0110 (A); 001 $\rightarrow$ 1010 ($-A$); 010 $\rightarrow$ 1001 ($A + B$); 011 $\rightarrow$ 1101 ($B - A$); 10x $\rightarrow$ 1001 ($\overline{A}$); 11x $\rightarrow$ 0101 ($A \oplus B$). All correct.

Gates: $G = \overline{S_2}S_0 + S_2\overline{S_1}$ is shared by all four stages (2 AND, 1 OR, 2 NOT); then each stage needs one XOR (X_i), one AND (Y_i) and one AND (Z_i).

Question 3 — IEEE 754 (3+3)

(a) What are the advantages of using biased exponent representation?

Concept: §2, representation • **Source:** COD §3.5

Solution.

- The exponent field is an **unsigned** number, so larger exponents are larger binary values.
- Two floating-point numbers can therefore be compared — and sorted — with an ordinary integer comparator: sign first, then the exponent and fraction together. No special FP compare hardware.
- The most negative exponent is all 0s, so zero is encoded as the all-zero word, and the exponent needs no separate sign handling in the adder's exponent-difference logic.

(b) What is the representation of zero and the representation of infinity?

Concept: §2, special values • Source: COD §3.5, Fig 3.13; Ch3 slides 17–18

Solution. Zero: exponent all 0s and fraction all 0s. The sign bit is free, so there are two zeros: $+0 = 0x00000000$ and $-0 = 0x80000000$ in single precision. **Infinity:** exponent all 1s and fraction all 0s: $+\infty = 0x7F800000$, $-\infty = 0xFF800000$. (Exponent all 1s with a non-zero fraction is NaN.)

Question 4 — 16-bit floating-point formats (3+3)

Question 4

For each format, determine the largest and smallest positive floating-point numbers. (a) Sign 1 bit, Exponent 4 bits, Mantissa 11 bits. (b) Sign 1 bit, Exponent 5 bits, Mantissa 10 bits.

Concept: §2, range and precision • Source: COD §3.5; Ch3 slides 12–13

Solution. Use $\text{Bias} = 2^{k-1} - 1$, smallest normalised $= 2^{1-\text{Bias}}$, largest $= (2 - 2^{-m}) \times 2^{(2^k-2)-\text{Bias}}$ (all-0s and all-1s exponents are reserved).

(a) $k = 4$, $m = 11$, **Bias** = 7. Smallest normalised: exponent 0001, $e = -6$, value $2^{-6} = \mathbf{0.015625}$. Largest: exponent 1110, $e = 7$, value $(2 - 2^{-11}) \times 2^7 = \mathbf{255.9375}$.

(b) $k = 5$, $m = 10$, **Bias** = 15. Smallest normalised: $e = -14$, value $2^{-14} \approx \mathbf{6.10 \times 10^{-5}}$. Largest: $e = 15$, value $(2 - 2^{-10}) \times 2^{15} = \mathbf{65504}$.

If denormals count, the smallest positive values are $2^{-11} \times 2^{-6} = 2^{-17} \approx 7.63 \times 10^{-6}$ and $2^{-10} \times 2^{-14} = 2^{-24} \approx 5.96 \times 10^{-8}$. State which assumption you use.

Comment: (a) has more precision but a far smaller range; (b), the IEEE half format, gives up one mantissa bit for one exponent bit and gains a range about 2^{16} times wider.

Question 5 — Pipelining and forwarding (3+3)

Question 5

Stage 1 (IF), 2 (ID), 3 (EX), 4 (MEM), 5 (WB); pipeline registers A (1–2), B (2–3), C (3–4), D (4–5). Initially $\$t1 = 10$, $\$t2 = 5$, $\$t3 = 0$, $\$t4 = 10$, $\$t5 = 0$. Inst1: `add $t3,$t1,$t2`; Inst2: `or $t4,$t1,$t2`; Inst3: `and $t2,$t1,$t2`; Inst4: `add $t5,$t3,$t1`. When Inst1 is in stage 5, Inst4 is in stage 2 of the same cycle. Ideally $\$t3$ is written in the first half and read in the second half, so Inst4 reads 15. A mistake causes a race: $\$t3$ is *not* updated in time, so Inst4 reads 0 (the *WB hazard*). Handle it by forwarding 15 to stage 2 of Inst4. (a) Show the pipelined execution of the 4 instructions and the forwarding path between Inst1 and Inst4. (b) Write the corresponding forwarding condition.

Concept: §5, forwarding • Source: COD §4.7, Figs 4.56, 4.60; Ch4 Part 2 slides 56–65

Solution. (a)

	CC1	CC2	CC3	CC4	CC5	CC6	CC7	CC8
Inst1 add \$t3	IF	ID	EX	MEM	WB			
Inst2 or \$t4		IF	ID	EX	MEM	WB		
Inst3 and \$t2			IF	ID	EX	MEM	WB	
Inst4 add \$t5				IF	ID	EX	MEM	WB

In CC5 Inst1 is in WB and Inst4 in ID. Draw the forwarding path from the **output of register D (MEM/WB)** — the write-back value **\$t3 = 15** — into **stage 2 of Inst4**, at the register-file read output (a multiplexer that chooses between the register file and the forwarded value).

(b)

```

if (MEM/WB.RegWrite and (MEM/WB.RegisterRd ≠ 0)
    and (MEM/WB.RegisterRd = IF/ID.RegisterRs)) ForwardA_ID = 1
if (MEM/WB.RegWrite and (MEM/WB.RegisterRd ≠ 0)
    and (MEM/WB.RegisterRd = IF/ID.RegisterRt)) ForwardB_ID = 1

```

Here MEM/WB.RegisterRd = \$t3 = IF/ID.RegisterRs, so ForwardA_ID = 1 and Inst4 receives 15. The comparison is against **IF/ID** (not ID/EX) because the consumer is three instructions behind and still in ID.

Question 6 — Datapath marking (6)

Question 6

In the given MIPS architecture, mark the datapath and control signals for your instruction, chosen by (last three digits of student ID) mod 3: 0 → **lw**, 1 → **sw**, 2 → **beq**. The wires carry red letters; write the letters of the active lines. Write the control signals that must be enabled, and for each mux (m1, m2, m3, m4) the control input (0/1).

Concept: §4 • Source: COD §4.4, Figs 4.17, 4.20, 4.21, 4.22; Ch4 Part 1 slides 26–30

Solution. A full-mark answer has three parts: the marked figure, the control signals, and m1–m4.

	m1	m2	m3	m4	Control signals asserted (all others 0 or X)
lw	0	0	1	1	RegWrite, MemRead, ALUSrc, MemtoReg; ALUOp = 00
sw	0	X	1	X	MemWrite, ALUSrc; ALUOp = 00
beq	B-Z	X	0	X	Branch; ALUOp = 01

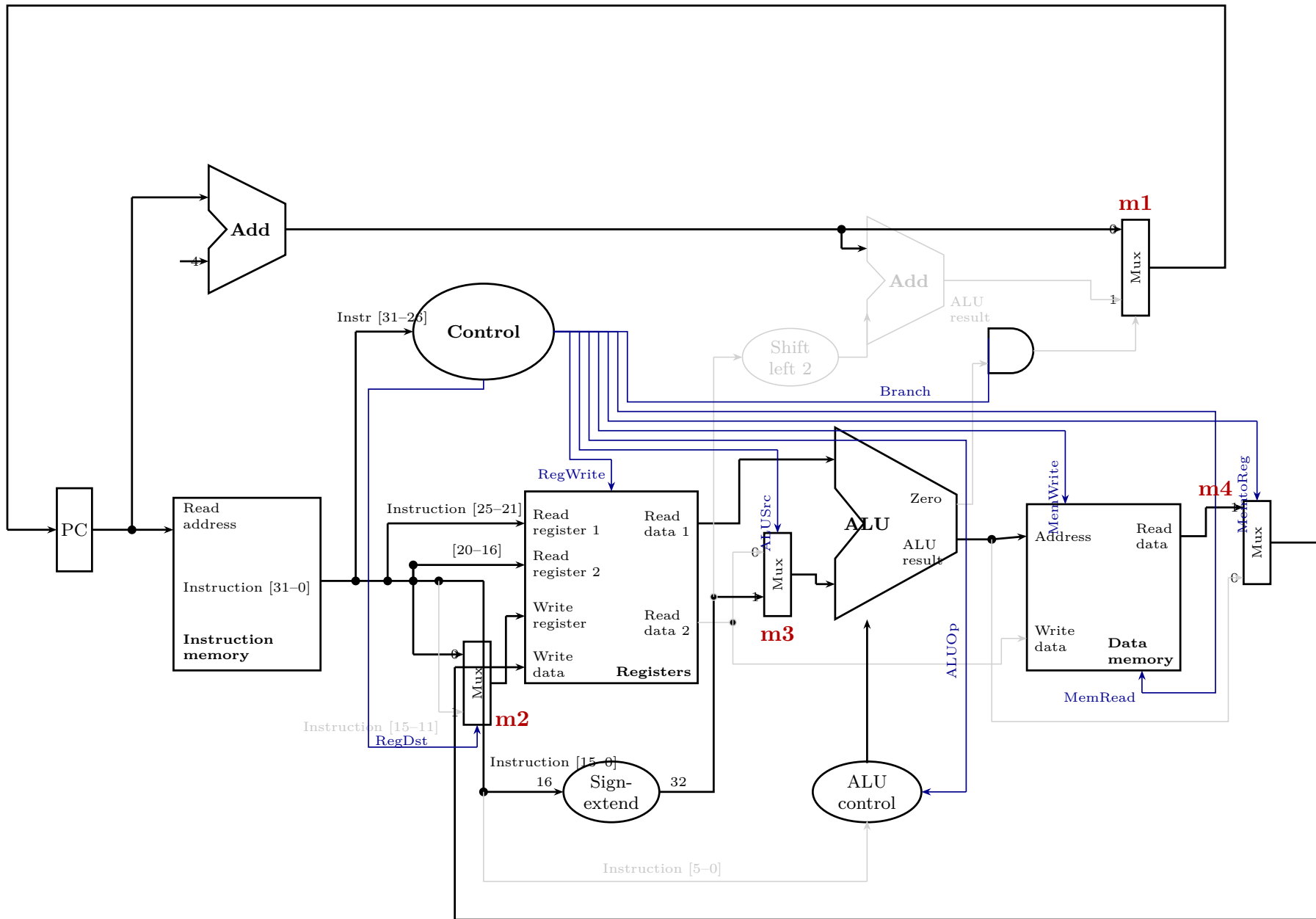
m1 = PCSrc, m2 = RegDst, m3 = ALUSrc, m4 = MemtoReg; B-Z = Branch AND Zero (1 only if taken).

Active wires, by name (Figures 7–9 show them marked):

- **lw**: PC → instruction memory; PC → +4 adder → m1 → PC; Instr[31–26] → Control; Instr[25–21] → Read register 1; Instr[20–16] → m2 input 0 → Write register; Instr[15–0] → sign-extend → m3 input 1; Read data 1 → ALU; m3 → ALU; ALU result → data-memory Address; memory Read data → m4 input 1 → register Write data.
- **sw**: PC → instruction memory and the +4 loop; Instr[31–26] → Control; Instr[25–21] → Read register 1; Instr[20–16] → Read register 2; Instr[15–0] → sign-extend → m3 input 1; Read data 1 → ALU; ALU result → Address; Read data 2 → memory Write data. No write-back.
- **beq**: PC → instruction memory and the +4 adder; Instr[31–26] → Control; Instr[25–21] and Instr[20–16] → both read ports; Read data 1 → ALU; Read data 2 → m3 input 0 → ALU; ALU Zero → AND gate; Instr[15–0] → sign-extend → shift-left-2 → branch adder; PC + 4 → branch adder; branch adder → m1 input 1; AND output → m1 select.

Letters. On the exam figure the letters sit on these wires (read from the scan; check them against your copy, since the scan is faint for *l*, *m*, *q* and *s*): a new PC → PC; b PC → +4 adder; c PC + 4 → m1; d PC + 4 → branch adder; e Instr[31–26]; f Instr[25–21]; g Instr[20–16] → Read reg 2; h Instr[20–16] → m2; i Instr[15–11] → m2; j Instr[15–0]; k Instr[5–0]; l sign-extend output; m Read data 2 → memory; n Read data 2 → m3; o Read data 1 → ALU; p m3 → ALU; q ALU control →

ALU; r ALU result $\rightarrow$ Address; s ALU result $\rightarrow$ m4; t memory Read data $\rightarrow$ m4; u m4 $\rightarrow$ Write data; v shift-left-2 $\rightarrow$ adder; w branch adder $\rightarrow$ m1. With that reading: **lw** = a, b, c, e, f, h, j, l, o, p, q, r, t, u; **sw** = a, b, c, e, f, g, j, l, m, o, p, q, r; **beq** = a, b, c, d, e, f, g, j, l, n, o, p, q, v, w.



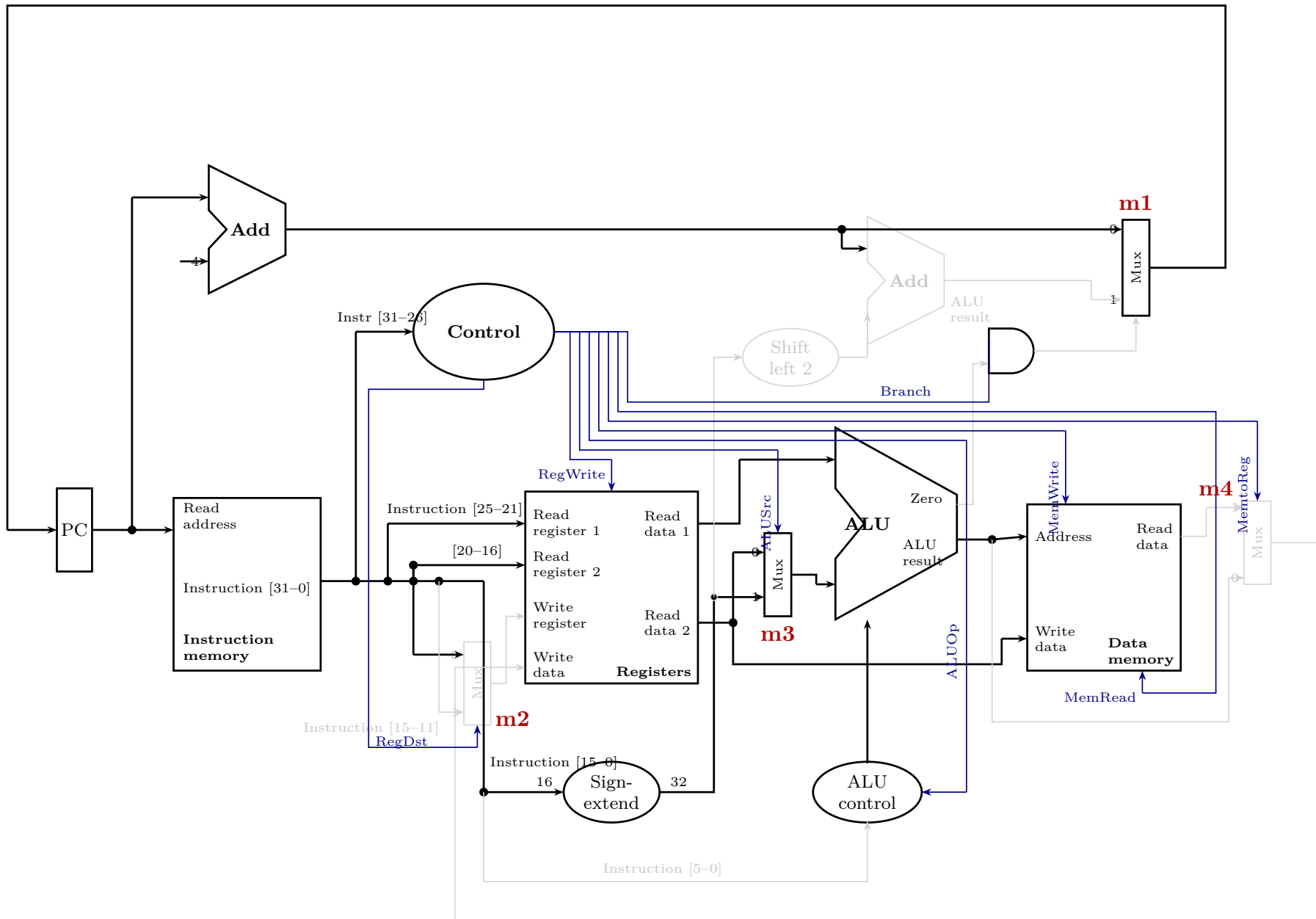


Figure 8: Question 6, sw: m1=0, m2=X, m3=1, m4=X. The write-back path is idle.

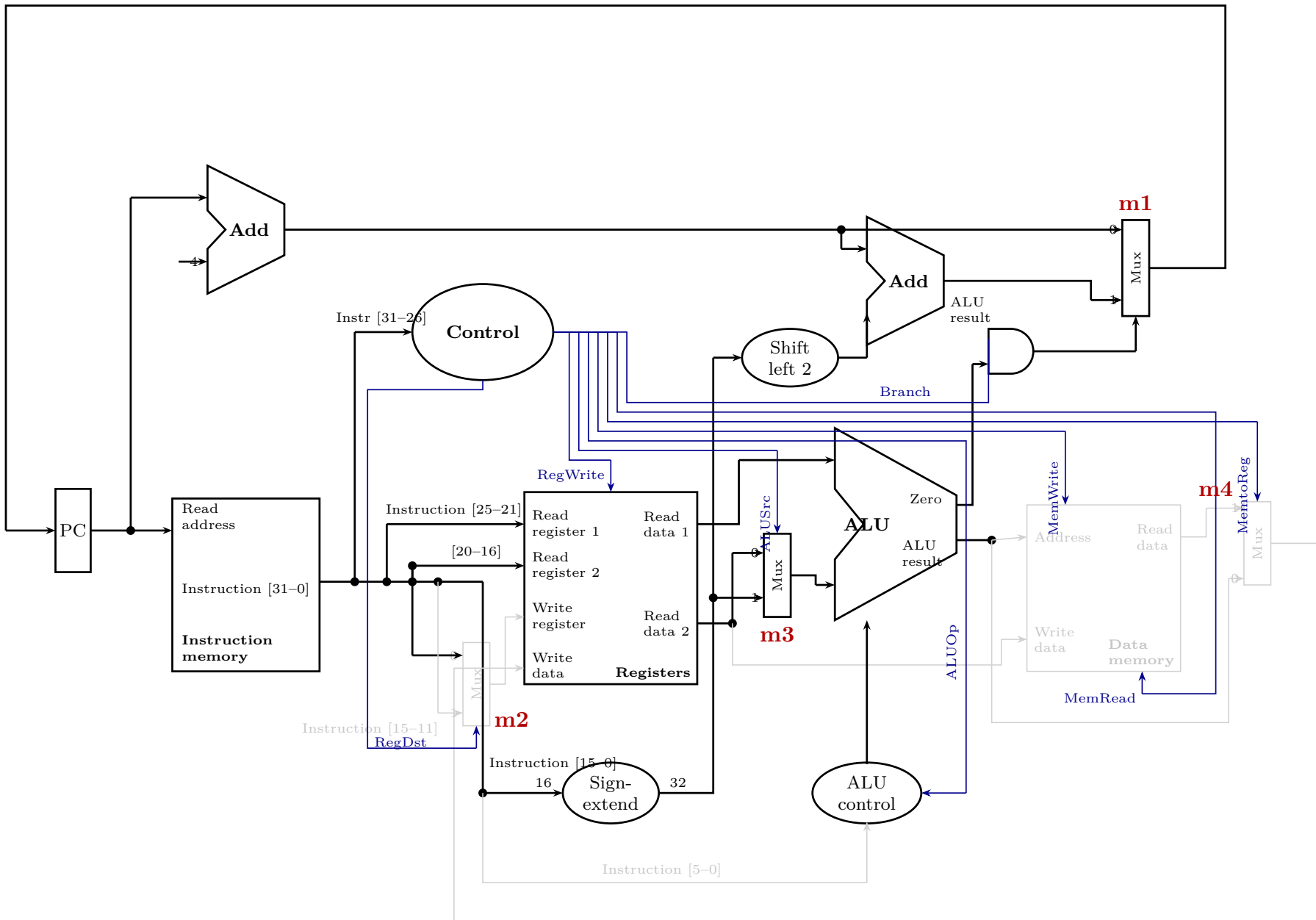


Figure 9: Question 6, beq: m1=Branch·Zero, m2=X, m3=0, m4=X. (COD Fig 4.21; Ch4 Part 1 slide 30.)

9 CSE 306 quiz, as recollected by a senior

About this paper

From the *Lab Quiz* notes in your folder: a senior's memory of a CSE 306 quiz from another batch — 10 MCQs and 5 fill-in-the-blanks (20 marks), then six short questions ($6 \times 5 = 30$). Question 3 was not remembered. The wording below is reconstructed from the notes.

Q1. If a conditional jump is many instructions away, how will you handle it? (The case in the notes was `bneq $t0, $t2, label3`.)

Concept: §3, addressing modes • **Source:** COD §2.10; Ch2 slide 54 (“Branching Far Away”)

Solution. A branch's 16-bit offset reaches only about $\pm 32K$ instructions. Invert the condition and branch *over* an unconditional jump, whose 26-bit field reaches anywhere in the 256 MB region:

```

    bneq $t0, $t2, label3      # target too far
# becomes
    beq  $t0, $t2, label_4     # inverted condition, short hop
    j    label3                # unconditional, long reach
label_4:
```

Q2. A 4-bit adder consists of four 1-bit adders. Design 1: the sum takes 2 gate delays and the carry 3. Design 2: the sum takes 3 and the carry 2. Which design should you choose?

Concept: §1, adders • **Source:** Mano Ch. 9 (parallel adder); ALU slides 3–5

Solution. Design 2. In a ripple-carry adder each stage waits for the previous carry, so the carry delay is paid once per stage and the sum delay only once at the end. Design 1: $4 \times 3 + 2 = 14$ gate delays. Design 2: $4 \times 2 + 3 = 11$. The faster carry wins, and the gap grows with the word width.

Q4. Design this ALU (the table in the notes, with C_{in} written over S_0): $S_1S_0 = 00$: 1's complement of A ; 01 : 1's complement of B ; 10 : $A \text{ XOR } B$; 11 : $A + B$ (OR). The senior's tip: “OR cannot have $C_{in} = 1$, so design Z_{in} accordingly.”

Concept: §1, the more efficient design • **Source:** Mano Table 9-3; ALU slide 17

Solution. The notes list two rows as 11; the only consistent reading is 00, 01, 10, 11 as above — say so in your answer. Every row is a *logic* operation, so the whole carry chain must be off: $F_i = X_i \oplus Y_i$.

S_1S_0	X_i	Y_i	$F_i = X_i \oplus Y_i$
00	A_i	1	$\overline{A_i}$
01	1	B_i	$\overline{B_i}$
10	A_i	B_i	$A_i \oplus B_i$
11	$A_i + B_i$	0	$A_i + B_i$

Simplifying the two columns:

$$X_i = A_i + S_0(\overline{S_1} + B_i)$$

$$Y_i = \overline{S_1}\overline{S_0} + (S_1 \oplus S_0)B_i$$

$$Z_i = 0$$

$Z_i = 0$ for every stage, including Z_1 : the external C_{in} (which shares the S_0 line) must *not* be connected to the adder, otherwise row 11 would compute $(A + B) + 1$ — exactly the senior's tip. Check ($A = 0110$, $B = 0011$): $00 \rightarrow 1001$, $01 \rightarrow 1100$, $10 \rightarrow 0101$, $11 \rightarrow 0111$. All correct.

Q5. Given an n -bit floating-point representation whose exponent is m bits long, show the steps for the

binary addition of $(0.4375)_{10}$ and $(0.5625)_{10}$.

Concept: §2, floating-point addition • **Source:** COD §3.5, Fig 3.14; Ch3 slides 19–20

Solution. Convert. $0.4375 = \frac{7}{16} = 111_2 \times 2^{-4} = 1.11_2 \times 2^{-2}$; $0.5625 = \frac{9}{16} = 1001_2 \times 2^{-4} = 1.001_2 \times 2^{-1}$. Bias = $2^{m-1} - 1$, so the stored exponents are Bias − 2 and Bias − 1.

Step 1, align: shift the smaller one: $1.11_2 \times 2^{-2} = 0.111_2 \times 2^{-1}$. **Step 2, add:** $0.111_2 + 1.001_2 = 10.000_2$, giving $10.000_2 \times 2^{-1}$. **Step 3, normalise:** $1.0000_2 \times 2^0$; exponent field = $0 + \text{Bias} = 2^{m-1} - 1$, i.e. $01 \dots 1$. No overflow or underflow. **Step 4, round:** nothing is lost. Result = 1.0, which checks: $0.4375 + 0.5625 = 1$.

Concretely in half precision ($n = 16$, $m = 5$, Bias 15): $0.4375 = 0\ 01101\ 1100000000 = 0x3700$, $0.5625 = 0\ 01110\ 0010000000 = 0x3880$, sum = $0\ 01111\ 0000000000 = 0x3C00$.

Q6. The datapath and control of MIPS are given. Mark the relevant paths and controls for the **bneq** instruction. (The senior's tip: you will most likely need to add new components.)

Concept: §4 • **Source:** COD §4.4, Figs 4.17, 4.21

Solution. **bneq** uses exactly the **beq** wires (Figure 9): both registers read, the ALU subtracts ($\text{ALUSrc} = 0$, $\text{ALUOp} = 01$), and the branch adder computes $PC + 4 + (\text{SE}(\text{imm}) \ll 2)$. But the branch must be taken when Zero = 0, and the given datapath only has Branch AND Zero. So **add hardware:** a new control signal **Bne**, an inverter on Zero and a second AND gate, and an OR in front of m1:

$$\text{PCSrc} = \text{Branch} \cdot \text{Zero} + \text{Bne} \cdot \overline{\text{Zero}}$$

Control for **bneq**: $\text{Bne} = 1$, $\text{Branch} = 0$, $\text{ALUSrc} = 0$, $\text{ALUOp} = 01$, $\text{RegWrite} = 0$, $\text{MemRead} = 0$, $\text{MemWrite} = 0$, RegDst and MemtoReg don't-care; $\text{m1} = 1$ when the registers differ, $\text{m3} = 0$. (An equally good design: XOR Zero with a new control bit before the existing AND gate.)

10 CT-1 (6 July 2026, 20 marks)

Q1. (10) Two processors, P1 and P2, run the same program of 10^9 instructions. P1 runs at 2 GHz with an average CPI of 2.0, and P2 runs at 3 GHz with an average CPI of 3.6. A proposed enhancement on P2 speeds up the floating-point instructions (20% of the program) by a factor of 4, but lowers P2's clock rate to 2.5 GHz. Determine whether the enhanced P2 is faster than P1, and by what factor.

Concept: §6 • **Source:** COD §1.6, §1.10 (Amdahl); Ch1 slides 19–26

Solution.

$$T_{P1} = \frac{10^9 \times 2.0}{2 \times 10^9} = 1.0 \text{ s}, \quad T_{P2} = \frac{10^9 \times 3.6}{3 \times 10^9} = 1.2 \text{ s}$$

The FP 20% of instructions drop from CPI 3.6 to $3.6/4 = 0.9$: $\text{CPI}_{\text{new}} = 0.8(3.6) + 0.2(0.9) = 3.06$.

$$T_{P2, \text{enh}} = \frac{10^9 \times 3.06}{2.5 \times 10^9} = 1.224 \text{ s}$$

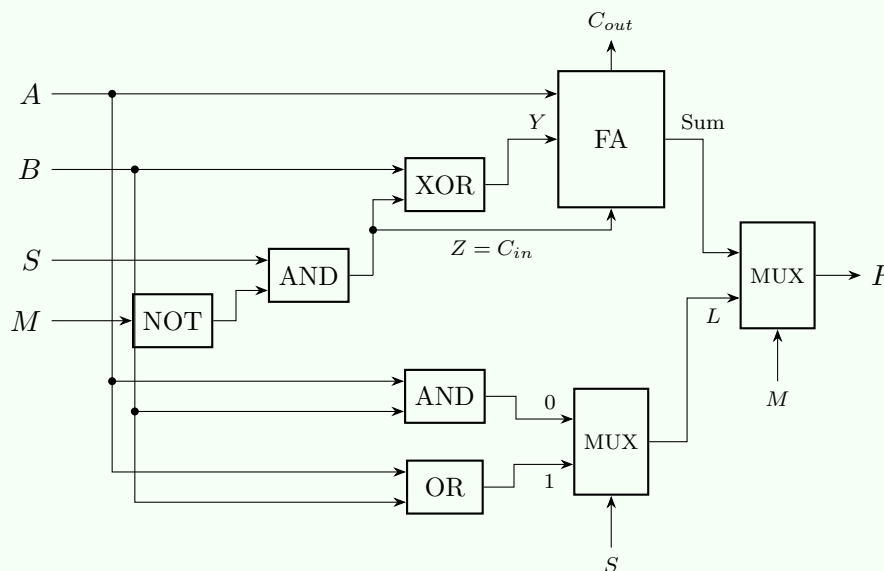
The enhanced P2 is **not** faster: P1 is faster by $1.224/1.0 = 1.224\times$. The enhanced P2 is even slower than the original P2 (1.2 s): a 15% CPI gain cannot pay for a 17% clock loss.

Q2. (10) Design a 1-bit ALU using only two control signals, M (mode) and S (selection). $M = 0$ selects arithmetic, $M = 1$ logic. Arithmetic: $S = 0$ addition ($A + B$), $S = 1$ subtraction ($A - B$). Logic: $S = 0$ AND, $S = 1$ OR. Draw the logic diagram of the 1-bit ALU using a full adder and basic logic gates.

Concept: §1, design example and combining • **Source:** Mano Figs 9-10, 9-12; ALU slides 12–16

Solution. Make the full adder an adder/subtractor (the lecture's design example), compute AND and OR beside it, and choose with multiplexers:

$$\begin{aligned}
 Z &= S \cdot \overline{M} && \text{carry-in: 1 only for subtraction} \\
 Y &= B \oplus Z && \text{complement } B \text{ only for subtraction} \\
 \text{Sum} &= A \oplus Y \oplus Z, \quad C_{out} = AY + AZ + YZ && \text{the full adder} \\
 L &= \overline{S}(AB) + S(A + B) && \text{logic result} \\
 F &= \overline{M} \cdot \text{Sum} + M \cdot L && \text{2-to-1 mux on M}
 \end{aligned}$$



Components: 1 full adder, 1 NOT, 1 AND (for Z), 1 XOR (for Y), 1 AND and 1 OR (logic), 2 two-to-one multiplexers. C_{out} is meaningful only when $M = 0$.

11 CT-2 (3 August 2026, 20 marks)

Question

The routine `evaluate` is the entry point. Study the call structure and the flow of values, then complete the five blanks (i)–(v). Each blank requires exactly one MIPS instruction.

```

# power : computes a^b (a in $a0, b in $a1)
power:
    addi $sp, $sp, -8
    sw   $ra, 4($sp)
    sw   $s0, 0($sp)
    ----- # (i)
    addi $v0, $zero, 1
    j     pow_end
pow_rec:
    ----- # (ii)
    addi $a1, $a1, -1
    jal  power
    mul  $v0, $v0, $s0
pow_end:
    lw   $s0, 0($sp)
    lw   $ra, 4($sp)
    addi $sp, $sp, 8
    jr   $ra

```

```

# fact : returns n!
fact:
    addi $sp, $sp, -8
    sw   $ra, 4($sp)
    sw   $s0, 0($sp)
    addi $v0, $zero, 1
    ----- # (iii)
    add $s0, $a0, $zero
    addi $a0, $a0, -1
    jal  fact
    mul  $v0, $v0, $s0
fact_end:
    lw   $s0, 0($sp)
    lw   $ra, 4($sp)
    addi $sp, $sp, 8
    jr   $ra

```

```

# evaluate : entry routine
evaluate:
    ----- # (iv)
    sw    $ra, 8($sp)
    sw    $s0, 4($sp)
    sw    $s1, 0($sp)
    add   $s0, $a1, $zero
    jal   fact
    add   $s1, $v0, $zero
    addi  $a0, $zero, 2
    ----- # (v)
    jal   power
    mul   $v0, $v0, $s1
    lw    $s1, 0($sp)
    lw    $s0, 4($sp)
    lw    $ra, 8($sp)
    addi  $sp, $sp, 12
    jr    $ra

```

Concept: §3, procedures and the stack • **Source:** COD §2.8, Figs 2.11, 2.12; Ch2 slides 36–44

Solution. (i) `bne $a1, $zero, pow_rec` — the base case $b = 0$ falls through to $\$v0 = 1$ and jumps to the epilogue, so the blank must send $b \neq 0$ to the recursive part.

(ii) `add $s0, $a0, $zero` — after the recursive call the code multiplies by $\$s0$, so a must be saved in a callee-saved register first; that is why $\$s0$ was pushed on entry.

(iii) `beq $a0, $zero, fact_end` — with $\$v0 = 1$ already set, $n = 0$ must skip the recursion.

(iv) `addi $sp, $sp, -12` — three words are saved at offsets 8, 4, 0, and the epilogue adds 12.

(v) `add $a1, $s0, $zero` — `power` is called as 2^b : $\$a0$ was set to 2, and b is the original second argument that was parked in $\$s0$.

So `evaluate(x, y)` returns $2^y \times x!$.

12 CT-3 (20 marks)

Question

```

1. lw    $t0, 0($s0)
2. sub   $t1, $t0, $s1
3. beq   $t1, $zero, L1
4. add   $t2, $t1, $s2
5. sw    $t2, 4($s0)
6. j     L2
L1:
7. add   $t2, $t0, $s3
8. sw    $t2, 8($s0)
L2:
9. or    $t4, $t1, $t2

```

Assumptions: 5-stage IF–ID–EX–MEM–WB; no code rescheduling; all-stage forwarding; branch resolved in ID; taken-branch penalty 1 cycle; load-use hazard 1 stall; unconditional jump penalty 1 cycle.

Determine: (a) all data hazards; (b) which cause stalls; (c) all control hazards; (d) total cycles if the branch at instruction 3 is (i) not taken, (ii) taken.

Concept: §5 • **Source:** COD §4.7–4.8; Ch4 Part 2 slides 10–32, 67–81

Solution. (a) **Data hazards (RAW):** I1→I2 ($\$t0$); I2→I3 ($\$t1$); I2→I4 ($\$t1$); I1→I7 ($\$t0$); I4→I5 ($\$t2$); I7→I8 ($\$t2$); I2→I9 ($\$t1$); I4 or I7 → I9 ($\$t2$).

(b) **Stalls:** I1→I2, load-use: the data exists only after MEM, the `sub` needs it in EX $\Rightarrow$ 1 stall. I2→I3: the branch compares in ID, but $\$t1$ is computed only at the end of I2's EX $\Rightarrow$ 1 stall, then forward EX/MEM → ID. Every other pair is covered by forwarding.

(c) **Control hazards:** the `beq` (I3), 1 cycle when taken; the `j` (I6), 1 cycle always.

(d) cycles = 4 + N + stalls + penalties. (i) Not taken: I1–I6 then I9, $N = 7$; stalls 2; jump 1 $\Rightarrow 4 + 7 + 2 + 1 = \mathbf{14}$. (ii) Taken: I1, I2, I3, I7, I8, I9, $N = 6$; stalls 2; taken branch 1 $\Rightarrow 4 + 6 + 2 + 1 = \mathbf{13}$.

If your teacher counts “all-stage forwarding” as removing the I2→I3 stall, the totals are 13 and 12. State the assumption — the reasoning earns the marks.

13 CT-4 (20 marks)

Question

1. A processor has a 32-bit address, a 64KB cache, a block size of 64 bytes and a 4-way set-associative organisation. (a) Number of cache blocks and sets. (4) (b) Bits for the tag, set index and block offset. (4) (c) For address 0xABCDEF12, the tag, set index and block offset. (4)
2. What are the advantages and disadvantages of using a large block? How can the disadvantages be addressed? (4+4)

Concept: §7 • Source: COD §5.3–5.4, Fig 5.15; Ch5 slides 26–28, 38–44

Solution. 1(a) Blocks = 64KB/64B = **1024**; sets = 1024/4 = **256**.

1(b) Offset = $\log_2 64 = \mathbf{6}$; index = $\log_2 256 = \mathbf{8}$; tag = $32 - 8 - 6 = \mathbf{18}$ bits.

1(c) 0xABCDEF12 = 1010 1011 1100 1101 1110 1111 0001 0010. From the right: offset = 010010 = 18; index = 10111100 = 188 = 0xBC; tag = 101010111100110111 = 0x2AF37.

2. Advantages: large blocks exploit spatial locality and need fewer tags per byte. Disadvantages: fewer blocks for the same capacity, so more conflict and capacity misses; a larger miss penalty because more bytes are transferred; and pollution by bytes that are never used. Remedies: early restart and critical-word-first, wider or interleaved memory to cut the transfer time, prefetching, and a victim cache to absorb conflicts.

14 Four mock papers

Built on the same pattern. Answers follow each set; the section sign points to where the topic is taught.

Mock paper 1

1. (9) MCQs: (i) bias for 6 exponent bits; (ii) V for a 5-bit ALU; (iii) the flags of $1100 + 0100$ in 4 bits; (iv) which of **sw**, **lw**, **beq** write no register; (v) exponent all 1s with fraction 0; (vi) ALUOp for R-type; (vii) which mux is selected by data rather than by the opcode; (viii) stalls for a load-use pair with forwarding; (ix) 2 GHz/CPI 1.6 versus 2.5 GHz/CPI 2.0, same program.
2. (12) Derive X_i , Y_i , Z_i : $s_2s_1s_0 = 000$ transfer A , 001 increment A , 010 add, 011 add with carry, 10x AND, 11x XOR.
3. (6) Encode -12.375 in half precision and decode it back.
4. (6) A 16-bit format is $1 + 6 + 9$: bias, largest and smallest positive normalised numbers.
5. (6) Mark the datapath for **lw \$t2, 12(\$t3)**: control signals and m1–m4.
6. (6) **add \$t1,\$t2,\$t3** followed by **sub \$t4,\$t1,\$t5**: pipeline diagram, forwarding path, condition.

Solution. 1. (i) 31; (ii) $C_5 \oplus C_6$; (iii) result 0000 with $Z = 1$, $C = 1$, $S = 0$, $V = 0$; (iv) **sw** and **beq**; (v) $\pm\infty$; (vi) 10; (vii) m1, since PCSrc = Branch·Zero; (viii) 1; (ix) a tie, $1.6/2 = 2.0/2.5 = 0.8$ ns per instruction. **2.** $X_i = A_i$; $Y_i = s_1B_i$ (rows 010, 011 and the XOR row); $Z_1 = \overline{s_2}s_0$, $Z_i = \overline{s_2}C_i$; AND needs its own 4-bit gate and a final mux selected when $s_2s_1 = 10$ (§1). **3.** 0xCA30 (§2, Practice 2.2). **4.** Bias 31; largest $(2 - 2^{-9}) \times 2^{31} \approx 4.29 \times 10^9$; smallest $2^{-30} \approx 9.31 \times 10^{-10}$. **5.**

RegDst 0, ALUSrc 1, MemtoReg 1, RegWrite 1, MemRead 1, MemWrite 0, Branch 0, ALUOp 00; m1 0, m2 0, m3 1, m4 1 (Figure 3). **6.** EX hazard: forward EX/MEM $\rightarrow$ ALU input A, ForwardA = 10; no stall, $4 + 2 = 6$ cycles (§5).

Mock paper 2

- (9) MCQs: (i) which format has the widest immediate/address field; (ii) why j needs no sign extension; (iii) which flag detects unsigned overflow; (iv) NaN versus infinity; (v) sets in a 4-way 32 KB cache with 32-byte blocks; (vi) where the improved design resolves branches; (vii) CPI of a perfect 5-stage pipeline; (viii) what the hidden bit saves; (ix) which control signal distinguishes `add` from `addi`.
- (12) Your own Assignment-1 table (Section C2, Group 5): derive X_i , Y_i , Z_i and count the gates.
- (6) Add 1.75 and -0.28125 showing all four steps.
- (6) Add the half-precision numbers `0x4A00` and `0x4100`.
- (6) Mark the datapath for `beq` and explain why m1 is not known at decode time.
- (6) Count stalls and cycles for `add $t1,$t2,$t3; lw $t4,0($t1); sub $t5,$t4,$t1; sw $t5,4($t1)`.

Solution. 1. (i) J-type (26 bits); (ii) the target is an unsigned word address, concatenated rather than added; (iii) C (V is for signed); (iv) both have exponent all 1s; fraction 0 is ∞ , non-zero is NaN; (v) 256; (vi) ID; (vii) 1; (viii) one bit of storage, i.e. one extra bit of precision; (ix) ALUSrc (and RegDst). **2.** $X_i = A_i$, $Y_i = (cs_1 \oplus cs_0)B_i + cs_1cs_0$, $Z_i = \overline{cs_0}C_i$, $Z_1 = \overline{cs_0}\overline{cs_2}$ (§1, Practice 1.5). **3.** 1.46875 (§2, Practice 2.7). **4.** `0x4B40` = 14.5 (Practice 2.8). **5.** Figure 5; m1 = Branch·Zero and Zero exists only after the ALU subtracts. **6.** One stall (`lw` $\rightarrow$ `sub`); 9 cycles (§5, Practice 5.2).

Mock paper 3

- (9) Nine MCQs of your choice from Question 1 of the CSE 306 paper, answered without looking.
- (12) A 1-bit ALU with M and S: $A + B$, $A - B$, $A \cdot B$, $\overline{A}$. Equations and one-stage diagram.
- (6) Encode 0.1 in single precision and explain why it is inexact.
- (6) Write the opcodes of `add`, `addi`, `lw`, `sw`, `beq`, `bne`, `j`, `jal`, `slt`, `ori`.
- (6) Mark the datapath for `sw $t0, -8($t1)` and list the active control signals.
- (6) Fill the control table for `add`, `lw`, `sw`, `beq`, `addi`, `slt`.

Solution. 2. $Z = S\overline{M}$, $Y = B \oplus Z$, $\text{Sum} = A \oplus Y \oplus Z$, $L = \overline{S}(AB) + S\overline{A}$, $F = \overline{M}\text{Sum} + ML$ — the CT-1 diagram with the OR replaced by a NOT on A. **3.** `0x3DCCCCCD` (Practice 2.4). **4.** R 000000/100000; 001000; 100011; 101011; 000100; 000101; 000010; 000011; R 000000/101010; 001101. **5.** Figure 4: ALUSrc 1, MemWrite 1, ALUOp 00, RegWrite 0, MemRead 0, Branch 0; m1 0, m3 1, m2 and m4 X. **6.** §4, Practice 4.1.

Mock paper 4

- (9) MCQs on flags: the flags of $0111 + 0001$, $1000 - 0001$, $1111 + 0001$, $0011 - 0101$ (C, S, Z, V each).
- (12) Derive X_i , Y_i , Z_i : $S_1S_0 = 00$ transfer A, 01 add, 10 subtract, 11 decrement A.
- (6) An 8-bit mini-float $1 + 3 + 4$: bias, range; then encode 2.75.
- (6) What is added to the datapath for `jr`? For `addi`?

5. (6) A loop branch taken 9 times then not taken: mispredictions per visit with a 1-bit and a 2-bit predictor.
6. (6) A program has CPI classes 1, 2, 3 at 50%, 30%, 20%; 10^9 instructions at 2 GHz. CPI and time.

Solution. 1. Practice 1.6. **2.** Practice 1.2. **3.** Bias 3; range 0.25 to 15.5; $2.75 = 10.11_2 = 1.011_2 \times 2^1$: exponent $1 + 3 = 4 = 100$, fraction $0110 \Rightarrow 0100110$. **4.** jr: Read data 1 as another PC-mux input plus a select signal; addi: nothing new. **5.** 1-bit: 2; 2-bit: 1. **6.** CPI 1.7; 0.85 s.

A One-page cheat sheet

Read this in the last half hour

- **ALU:** $F_i = X_i \oplus Y_i \oplus Z_i$. $X_i = A_i \oplus (\text{complement condition})$; $Y_i \in \{0, B_i, \overline{B_i}, 1\}$; $Z_i = (\text{logic mode}) \cdot C_i$ — kill every carry in logic mode.
- **Recipes:** A ($Y=0, c=0$); $A+1$ ($Y=0, c=1$); $A+B$ ($Y=B, c=0$); $A+B+1$ ($Y=B, c=1$); $A-B-1$ ($Y=\overline{B}, c=0$); $A-B$ ($Y=\overline{B}, c=1$); $A-1$ ($Y=1s, c=0$); XOR ($Y=B, Z=0$); NOT ($Y=1, Z=0$).
- **Flags:** C carry out ($A-B$: $C=1 \Leftrightarrow A \geq B$); S = MSB; Z = NOR of all bits; V = carry into MSB $\oplus$ carry out. After AND/OR/XOR clear C and V.
- **FP:** Bias $2^{k-1} - 1$; $x = (-1)^S(1+f)2^{E-\text{Bias}}$; exponent 0s $\Rightarrow$ 0/denormal, 1s $\Rightarrow$ ∞ /NaN; half 1+5+10, bias 15, max 65504. Addition: align, add, normalise, round.
- **Control** (RegDst, ALUSrc, MemtoReg, RegWrite, MemRead, MemWrite, Branch, ALUOp): R 1,0,0,1,0,0,0,10; lw 0,1,1,1,0,0,0,0; sw X,1,X,0,0,1,0,0; beq X,0,X,0,0,0,1,01. m1 PCSrc, m2 RegDst, m3 ALUSrc, m4 MemtoReg.
- **ALU control:** 00 add, 01 subtract, 10 funct: add 0010, sub 0110, AND 0000, OR 0001, slt 0111.
- **Opcodes:** R 000000; lw 100011; sw 101011; beq 000100; bne 000101; j 000010; jal 000011; addi 001000; ori 001101.
- **Pipeline:** cycles = $4 + N + \text{stalls} + \text{penalties}$; load-use 1 stall; ALU result into a branch in ID 1 stall; taken branch in ID 1 cycle.
- **Performance:** time = $\text{IC} \times \text{CPI}/f$; Amdahl $1/((1-f) + f/k)$.
- **Cache:** offset $\log_2(\text{block})$, index $\log_2(\text{sets})$, tag the rest.